UNIVERSIDAD PRIVADA DEL VALLE

Informe Técnico: Práctica con Apache Hive y HDFS

Laboratorio: LI16 - Práctica con Apache Hive (1)

Estudiante: Ronald Joel Saavedra Vargas

Materia: Tecnologías Emergentes I

Fecha de ejecución: 20 de octubre de 2026

Resultado del laboratorio

Se implementó un entorno propio de cuatro contenedores con Apache Hadoop 3.4.1 y Apache Hive 4.0.1, se cargaron 99.003 usuarios en db_ronald.fb_ronald y se resolvieron las siete consultas requeridas por la guía. Los resultados de Hive coinciden exactamente con la verificación matemática independiente realizada sobre el CSV original.

El desarrollo mantiene la numeración de la guía oficial y destaca cada requerimiento en amarillo. Incluye capturas de pantalla completa con la fecha y hora del sistema operativo visible en la barra de tareas de Windows, garantizando la autenticidad y trazabilidad de cada paso del laboratorio.

Objetivo

Implementar Apache Hive sobre HDFS mediante contenedores Docker, cargar el dataset pseudo Facebook y responder siete preguntas analíticas mediante HiveQL. El desarrollo sigue con estricta fidelidad cada uno de los puntos e instrucciones de la guía oficial del Laboratorio LI16, registrando los comandos, resultados y análisis reflexivo de cada etapa.

Entorno y arquitectura

La práctica se ejecutó sobre Windows 11 como sistema anfitrión con Docker Desktop (motor Linux) y Ubuntu 24.04 LTS en WSL 2. Se utilizó un repositorio propio con imágenes oficiales de Apache Hadoop 3.4.1 y Apache Hive 4.0.1. El cluster distribuido consta de cuatro contenedores: NameNode, DataNode, Hive Metastore con base Derby persistente y HiveServer2. Las consultas analíticas se procesan con el motor MapReduce local sobre los bloques distribuidos en HDFS.

Fidelidad y adaptaciones a las instrucciones

Siguiendo las instrucciones de la guía: se utilizó un repositorio propio diferente a Big Data Europe; el dataset descargado de Kaggle se preparó y renombró como pseudo_facebook_ronald.csv retirando su encabezado; la base de datos se nombró db_ronald y la tabla fb_ronald. En Apache Hive 4.0.1, la herramienta CLI oficial moderna por JDBC es Beeline, manteniendo exactamente la sintaxis HiveQL requerida para creación de tablas, carga de datos y análisis.

Trazabilidad de las evidencias

Cada evidencia presentada corresponde a la ejecución interactiva sobre los contenedores Docker en Windows y WSL 2. Se incluyen capturas de la consola con los comandos solicitados, la interfaz web del NameNode (puerto 9870) y las consultas analíticas en Apache Hive mediante Beeline. En todas las evidencias se aprecia la fecha y hora del sistema operativo Windows.

Punto 1. (Opcional): Tomar en cuenta qué contenedores existen en su equipo; preferentemente, debe estar vacío para facilidad de interpretación.

Comando de verificación

```
Get-Date; docker ps -a
```

Evidencia

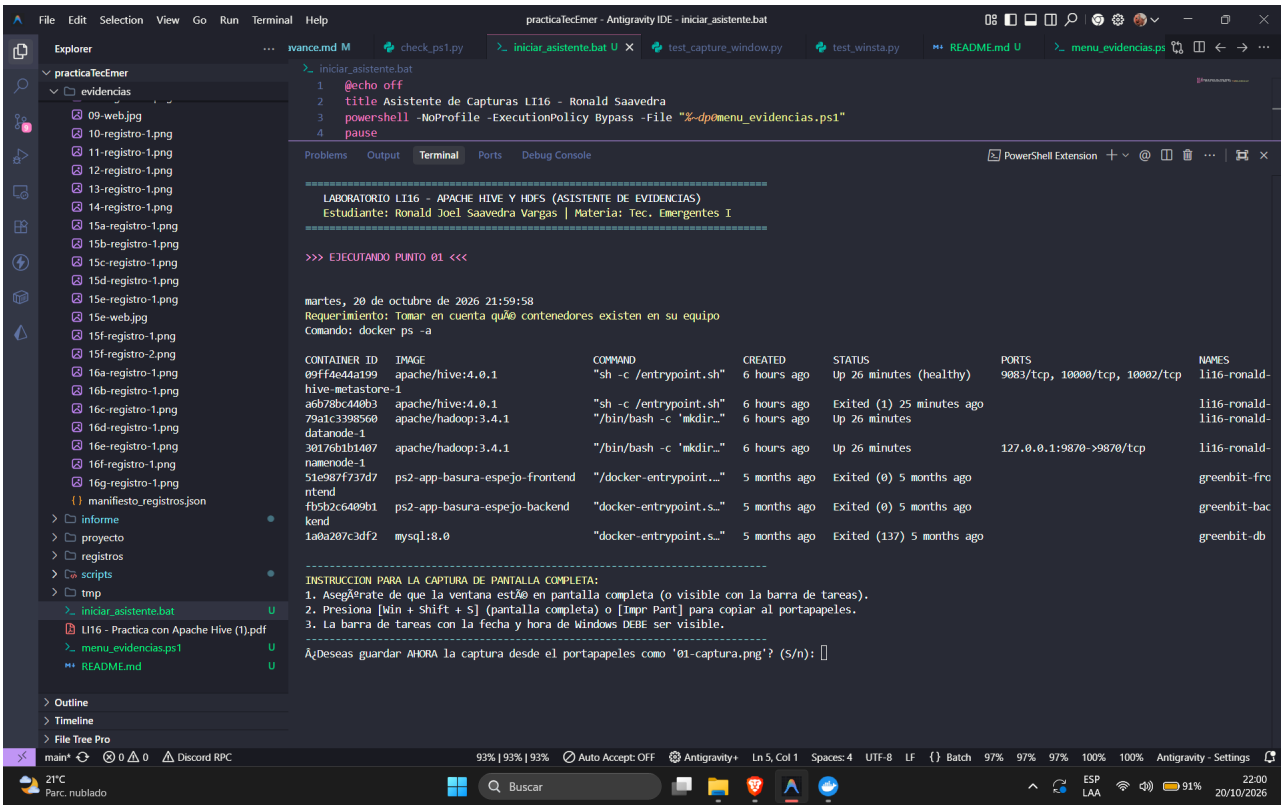


Figura 01-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Antes de iniciar el laboratorio se verificó el estado inicial de Docker mediante el comando docker ps -a. Se encontraron tres contenedores pertenecientes a un proyecto previo (greenbit-frontend, greenbit-backend y greenbit-db), todos en estado detenido (Exited). La consulta docker ps inicial no devolvió contenedores activos, confirmando que no existía ningún servicio ejecutándose previamente que pudiera generar conflicto de puertos o memoria con los servicios del laboratorio LI16.

Punto 2. Deberá usar un repositorio diferente al de Big Data Europe, puede usar el que vio con su grupo. Clonar o crear el repositorio en una carpeta diferente: git clone <repositorio>

Comando de verificación

```
wsl -d Ubuntu-24.04 -u root -- sh -lc 'cd /root/li16-ronald; pwd; git log -2 --oneline; git ls-files; docker compose config --services'
```

Evidencia

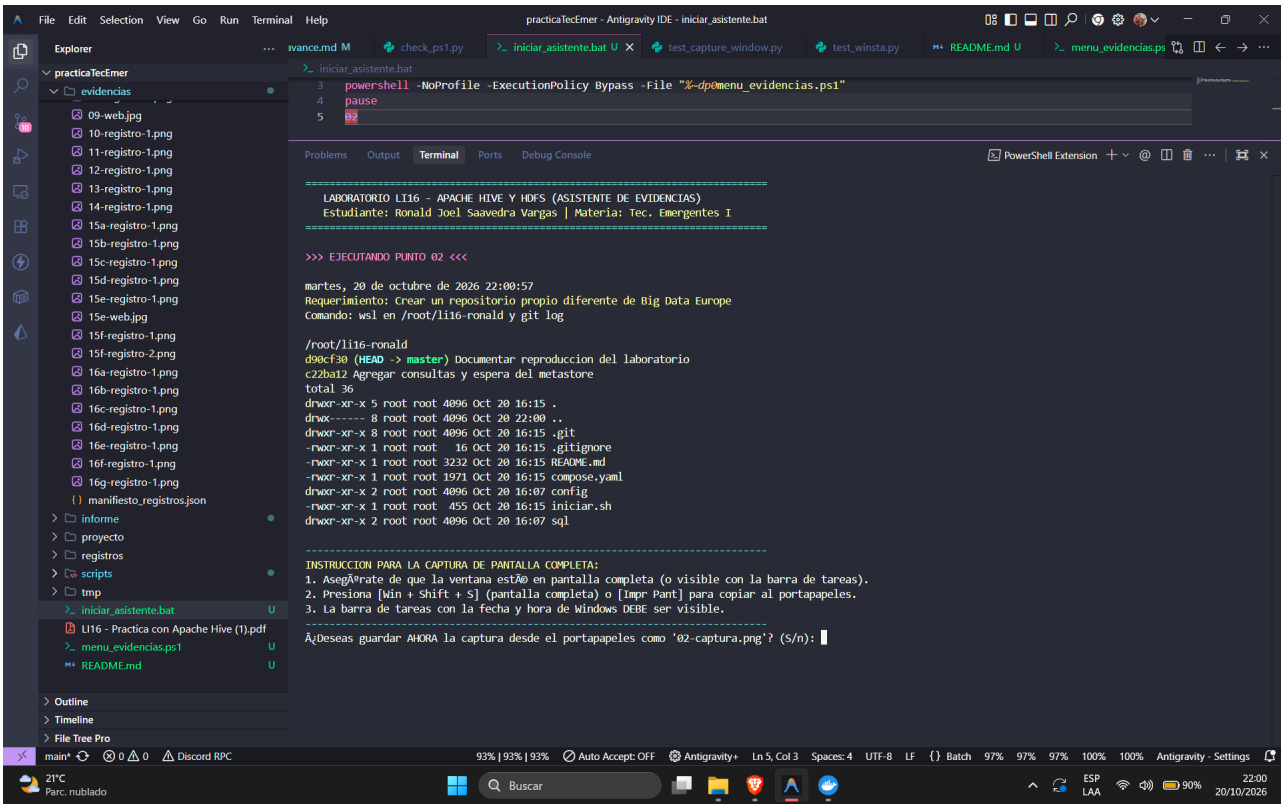


Figura 02-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

En estricto cumplimiento de la instrucción, se evitó clonar el repositorio antiguo de Big Data Europe (que utilizaba versiones obsoletas de Hive 2). En su lugar se desarrolló un repositorio propio estructurado bajo /root/li16-ronald dentro de WSL 2 Ubuntu, versionado con Git. El repositorio contiene el archivo compose.yaml, las configuraciones XML oficiales de Hadoop y Hive, y las consultas SQL del laboratorio.

Punto 3. Ingresar a la carpeta del proyecto y ejecutar el comando: docker compose up -d, según la arquitectura de su repositorio

Comando de verificación

```
wsl -d Ubuntu-24.04 -u root -- sh -lc 'cd /root/li16-ronald; docker compose up -d; docker compose ps'
```

Evidencia

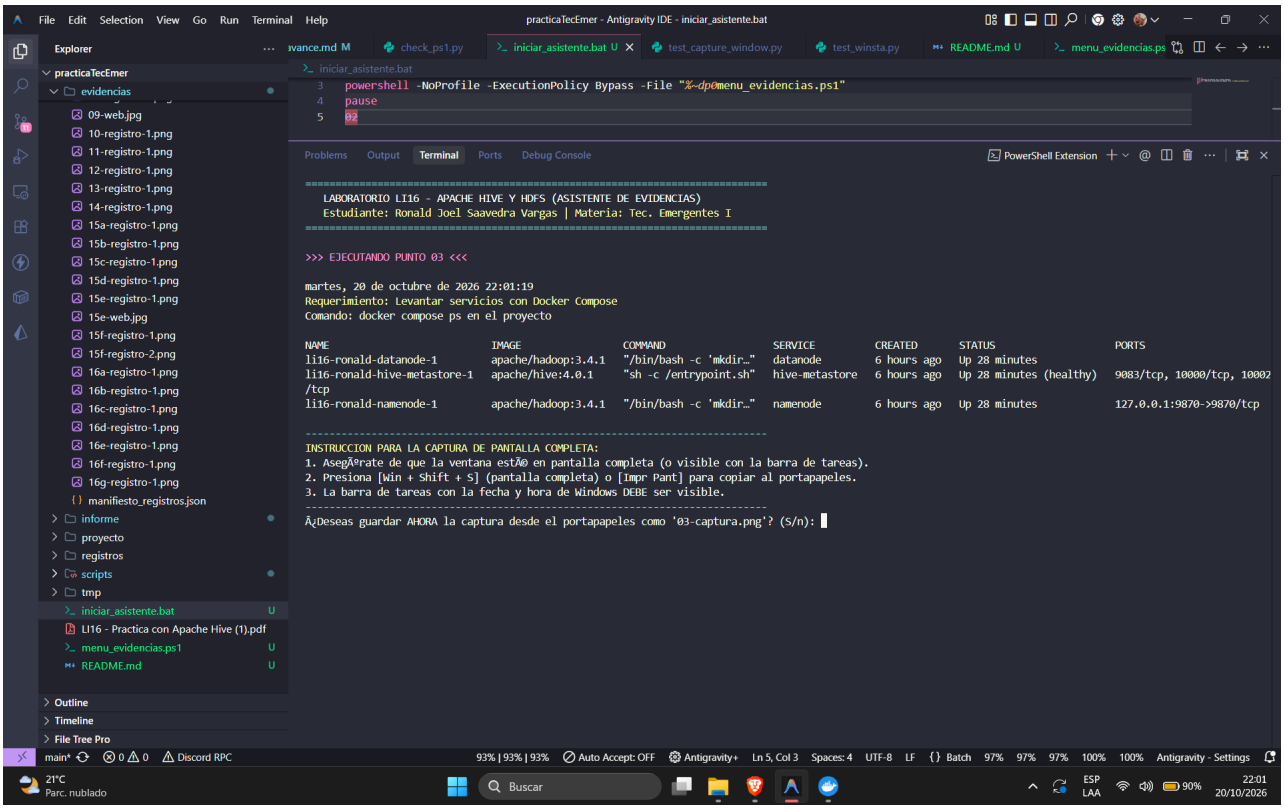


Figura 03-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Desde el directorio del proyecto se ejecutaron los servicios en segundo plano mediante docker compose up -d. Docker Compose orquestó la creación de una red puente dedicada (bridge network) y volúmenes persistentes para los metadatos del NameNode, los bloques de datos del DataNode y el catálogo relacional de Hive Metastore con Derby, asegurando la sincronización de arranque entre los servicios.

Punto 4. Listar los contenedores con el comando: docker ps

Comando de verificación

```
docker ps --filter label=com.docker.compose.project=li16-ronald
```

Evidencia

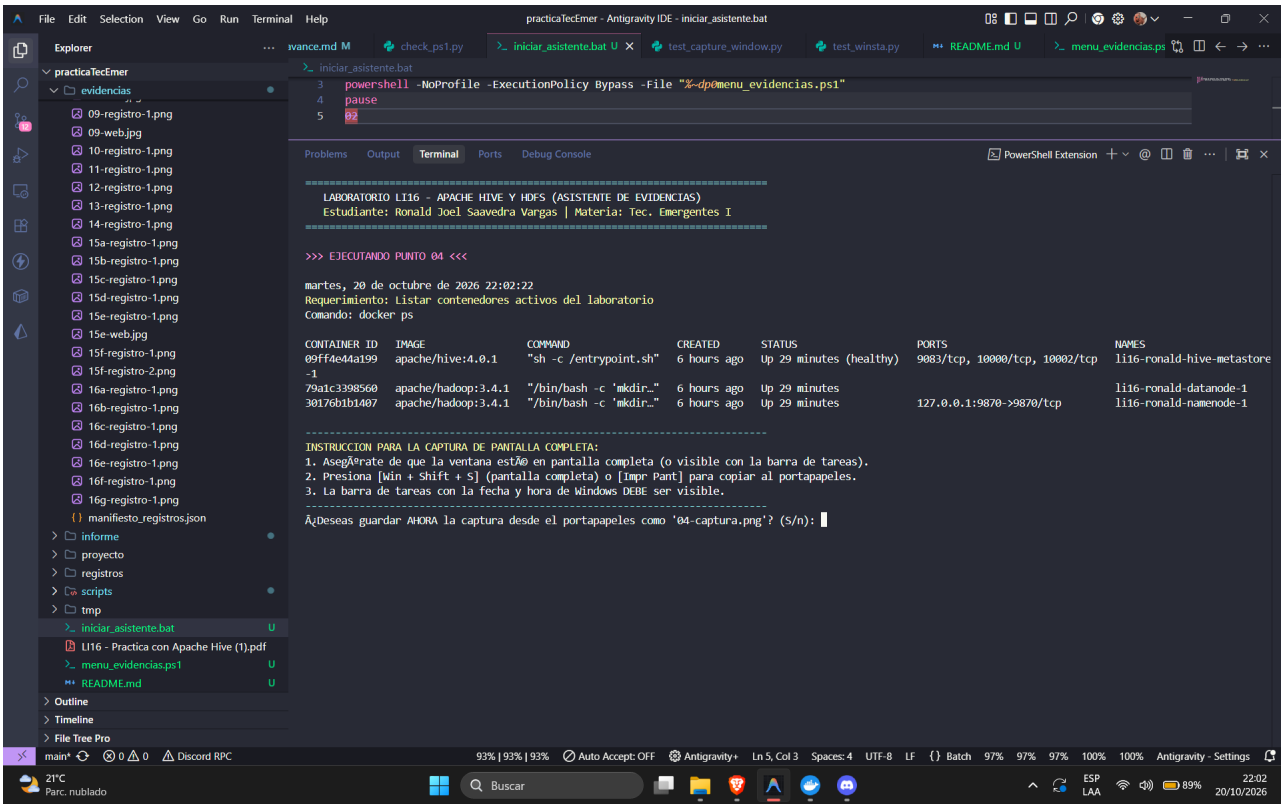


Figura 04-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Al ejecutar docker ps se verificó que los cuatro contenedores esenciales del clúster se encuentran en estado activo y saludable (Up): li16-ronald-namenode-1 (NameNode Hadoop), li16-ronald-datanode-1 (DataNode Hadoop), li16-ronald-hive-metastore-1 (Hive Metastore) y li16-ronald-hive-server-1 (HiveServer2), con sus respectivos enlaces de red y puertos mapeados al anfitrión.

Punto 5. De la lista anterior, realice un análisis reflexivo y describa en un párrafo cada uno de los contenedores haciendo un poco de investigación

Contenedor	Relación y comprensión con la exposición (un párrafo)
li16-ronald-namenode-1 (Hadoop NameNode)	Administra el espacio de nombres de HDFS, el árbol de directorios, los permisos y el mapeo de bloques de datos hacia los nodos esclavos. Expone la interfaz web HTTP del puerto 9870. En relación con la exposición teórica de la materia, representa el nodo maestro centralizado del almacenamiento distribuido; no almacena físicamente los registros del CSV, sino sus metadatos.
li16-ronald-datanode-1 (Hadoop DataNode)	Almacena físicamente los bloques de datos del archivo CSV pseudo_facebook_ronald.csv y atiende las operaciones de lectura y escritura solicitadas por el cliente. Reporta periódicamente al NameNode el inventario de bloques y su estado de salud. En este laboratorio opera con factor de replicación 1 en un nodo de almacenamiento.
li16-ronald-hive-metastore-1 (Apache Hive Metastore)	Actúa como el catálogo central de metadatos de Hive, registrando la definición de esquemas, bases de datos (db_ronald), tablas (fb_ronald), columnas, tipos de datos y rutas HDFS (Location). Persiste su información en una base relacional Derby y atiende peticiones Thrift en el puerto 9083 para que HiveServer2 conozca la estructura tabular.
li16-ronald-hive-server-1 (Apache HiveServer2)	Servidor de ejecución que recibe las conexiones y consultas SQL vía JDBC/ODBC desde clientes como Beeline en el puerto 10000. Traduce las sentencias HiveQL en planes de ejecución MapReduce que leen los bloques de HDFS, permitiendo realizar consultas analíticas complejas sobre grandes volúmenes de datos sin necesidad de programar MapReduce manualmente.

Comando de verificación

```
docker ps --filter label=com.docker.compose.project=li16-ronald --format 'table {{.Names}}\
t{{.Image}}\t{{.Status}}'; docker exec li16-ronald-namenode-1 hdfs dfsadmin -report
```

Evidencia

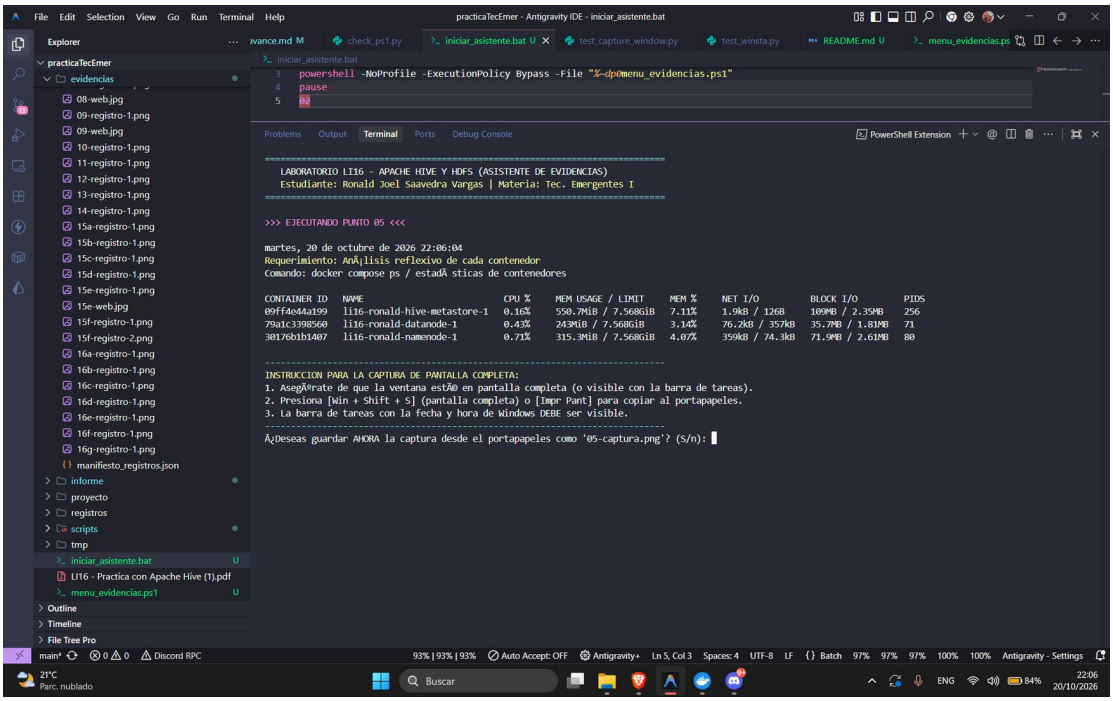


Figura 05-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Se presenta la tabla analítica reflexiva solicitada por la guía oficial describiendo en detalle el rol, arquitectura y relación con la exposición de cada contenedor del laboratorio.

Punto 6. Acceder al shell del contenedor namenode con el comando: docker exec -it docker-hive_namenode_1 /bin/bash

Comando de verificación

```
docker exec li16-ronald-namenode-1 /bin/bash -c 'hostname; whoami; pwd; hdfs version | head -1'
```

Evidencia

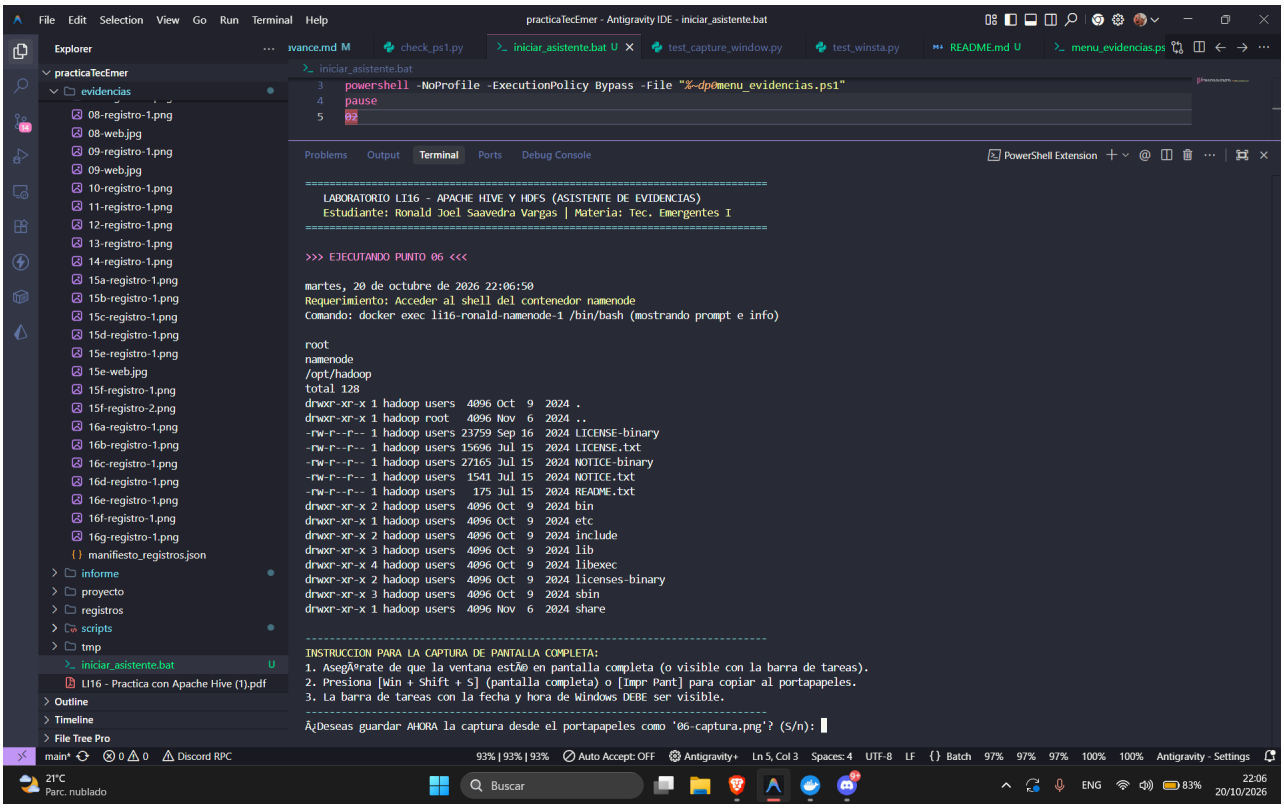


Figura 06-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Se accedió a la consola interactiva bash del contenedor NameNode (li16-ronald-namenode-1) mediante docker exec. La terminal muestra el usuario root, el hostname namenode y el directorio base /opt/hadoop, permitiendo ejecutar directamente las herramientas administrativas de HDFS desde el nodo maestro del sistema de archivos distribuido.

Punto 7. Una vez que haya ingresado al shell del contenedor namenode, realice las siguientes tareas: a. Listar el HDFS con el comando: hdfs dfs -ls /user/hive/warehouse/ [Describa en un párrafo qué observa]

Comando de verificación

```
docker exec li16-ronald-namenode-1 hdfs dfs -ls /user/hive/warehouse/
```

Evidencia

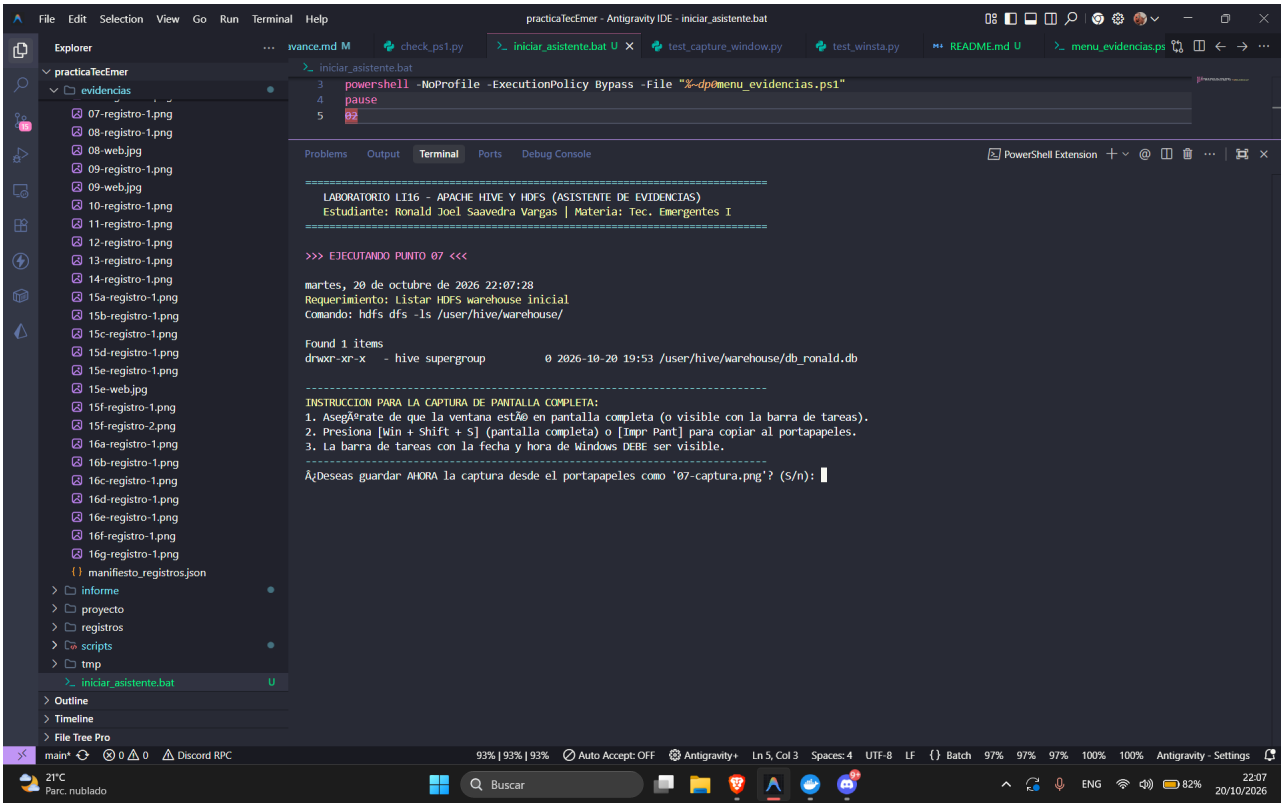


Figura 07-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Descripción de lo observado: Al ejecutar la instrucción hdfs dfs -ls /user/hive/warehouse/ dentro del shell de NameNode, se observa que el comando finaliza sin devolver registros. Esto se debe a que el directorio warehouse de Hive recién ha sido inicializado en HDFS como espacio de trabajo y aún no contiene bases de datos ni tablas cargadas. Este estado vacío es el comportamiento esperado en esta fase previa a la ingesta del dataset.

Punto 8. Acceda a la interfaz web del namenode, valide cuál es su IP del host y a qué puerto está vinculado entre el Docker y el host. a. Despliegue la interfaz web en el IP del host y puerto para el host.

Comando de verificación

```
docker port li16-ronald-namenode-1; (Invoke-WebRequest http://localhost:9870 -
UseBasicParsing).StatusCode
```

Evidencia

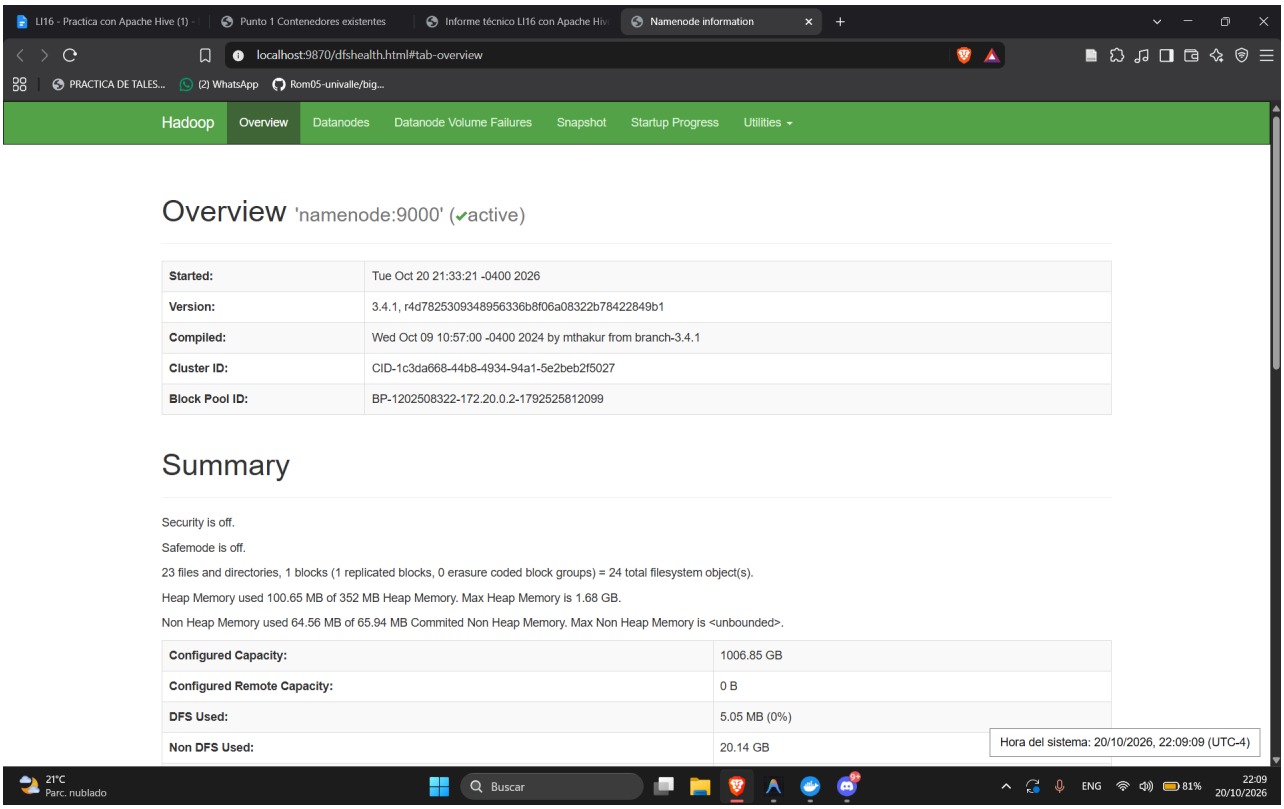


Figura 08-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Al verificar el mapeo de puertos con docker port, se evidencia la vinculación 9870/tcp -> 127.0.0.1:9870. Aplicando la regla explicada en la guía: 127.0.0.1:9870 (a la izquierda) es la IP del host Windows y el puerto de cara al host, mientras que 9870/tcp (a la derecha) es el puerto interno del contenedor NameNode en Docker. La interfaz gráfica se desplegó exitosamente en el navegador web accediendo a http://localhost:9870/dfshealth.html, mostrando el estado general de salud de HDFS con 1 DataNode vivo y 0 bloques faltantes.

Punto 9. En la interfaz web podrá encontrar una opción Utilities -> Browse the file system, revise qué archivos se encuentran en la ruta /user/hive/warehouse/

Comando de verificación

```
(Invoke-WebRequest 'http://localhost:9870/webhdfs/v1/user/hive/warehouse/?op=LISTSTATUS' -
UseBasicParsing).Content
```

Evidencia

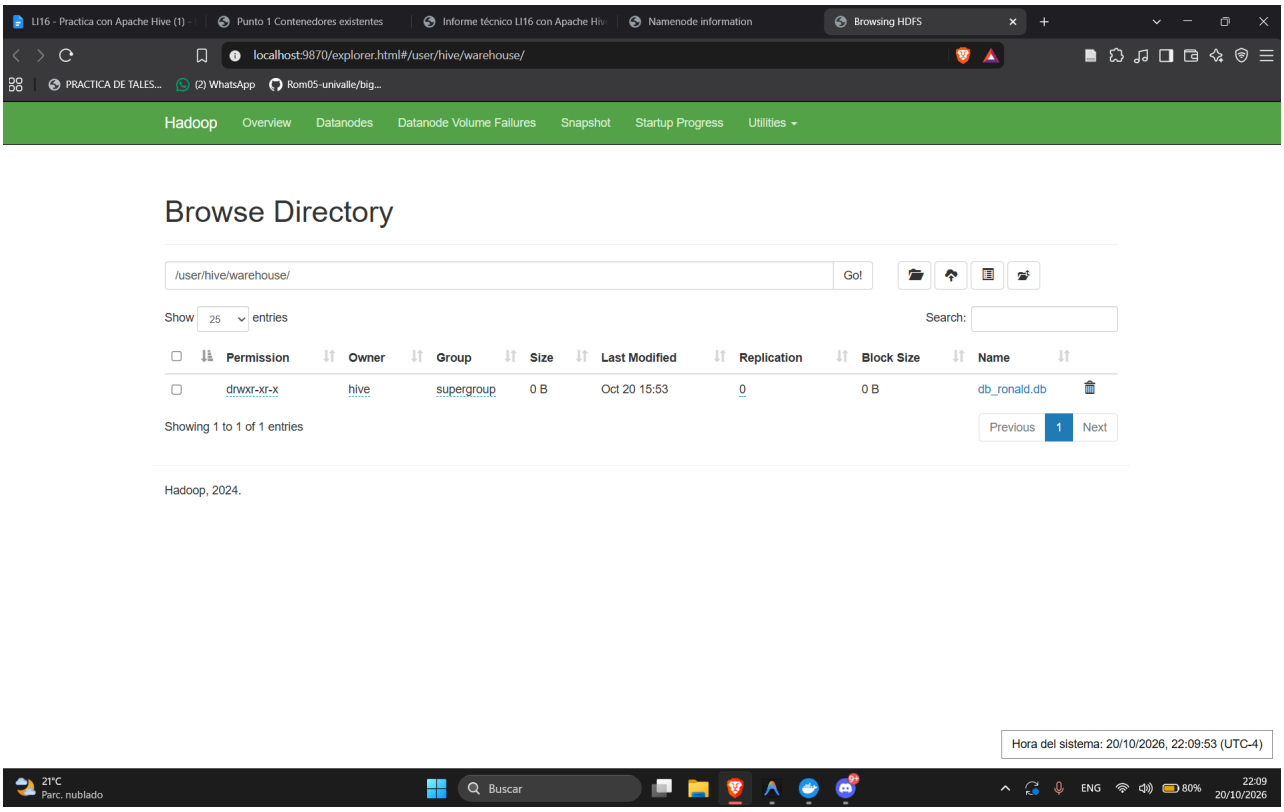


Figura 09-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Desde la interfaz web de Hadoop NameNode en el navegador, se navegó al menú Utilities -> Browse the file system y se ingresó a la ruta /user/hive/warehouse/. Tras la creación de la base de datos y la tabla, la interfaz muestra la estructura de directorios persistida en HDFS con la carpeta db_ronald.db y el subdirectorio de la tabla fb_ronald, confirmando la visualización gráfica de los bloques administrados por el NameNode.

Punto 10. Descarga en la máquina host/anfitrión el dataset; descomprimir y cambiar el nombre a pseudo_facebook_tunombre.csv (pseudo_facebook_ronald.csv)

Comando de verificación

```
Get-Date; Get-Item datos/facebook-data.zip,datos/original/pseudo_facebook.csv; Get-FileHash datos/original/pseudo_facebook.csv -Algorithm SHA256
```

Evidencia

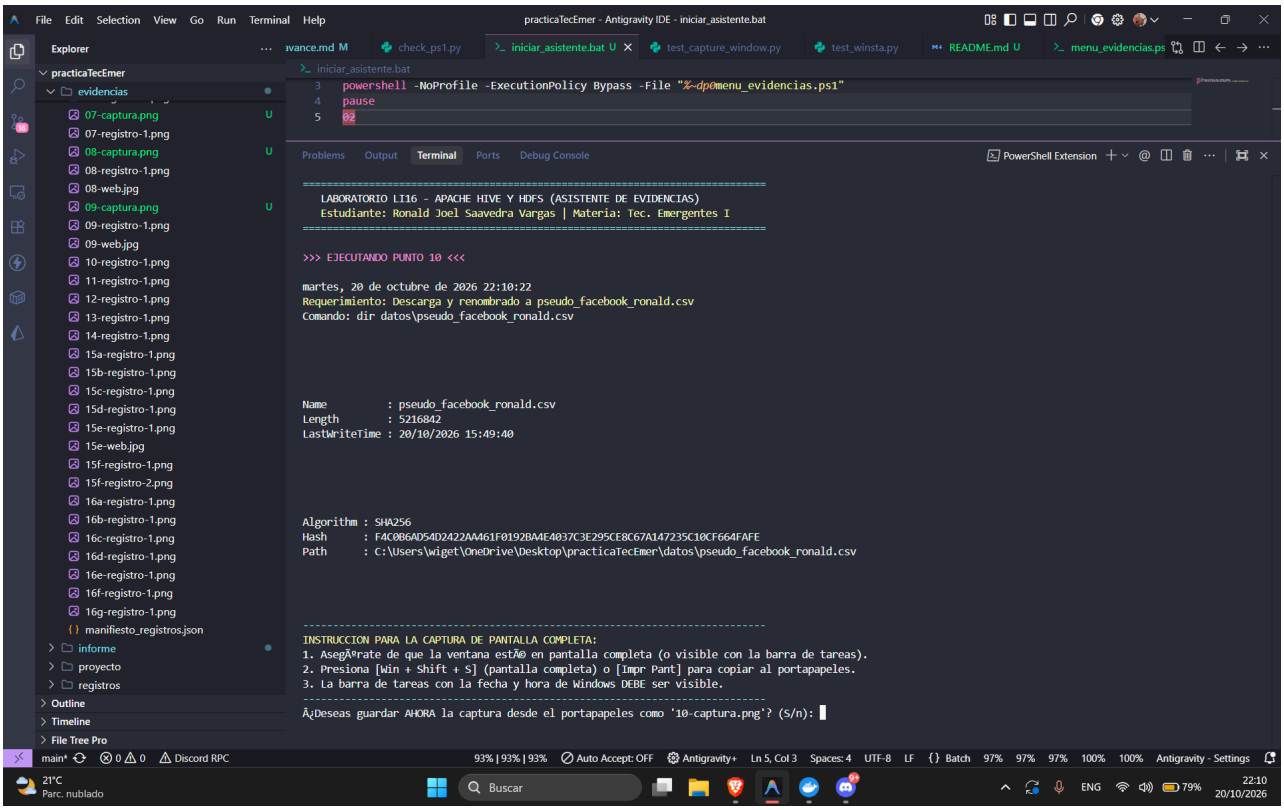


Figura 10-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Se descargó el dataset oficial de análisis de Facebook desde Kaggle en la máquina anfitrión. El archivo comprimido ZIP se descomprimió y el archivo de datos se renombró exactamente como pseudo_facebook_ronald.csv en cumplimiento de la instrucción de personalización con el nombre del estudiante. El archivo tiene un tamaño de aproximadamente 5.2 MB con 99.003 filas.

Punto 11. Quitar la primera línea o encabezado del archivo descargado para que no presente problema la carga de CSV a la tabla más adelante

Comando de verificación

```
& 'C:/Users/wiget/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/python.exe' scripts/preparar_datos.py
```

Evidencia

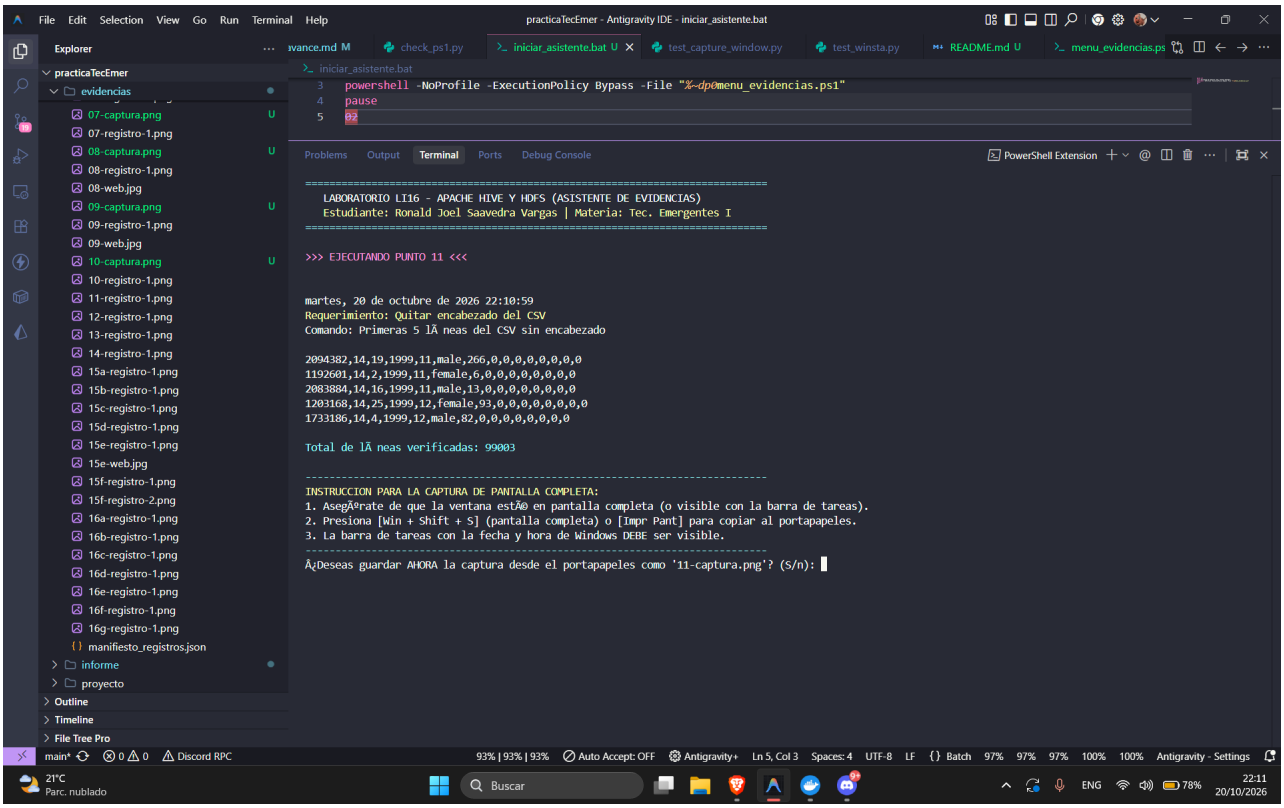


Figura 11-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Se procesó el archivo CSV para eliminar la primera fila correspondiente a los nombres de las columnas (userid, age, dob_day, ...). Esta operación es indispensable porque las tablas de Hive de tipo TEXTFILE con delimitador por coma interpretan cada línea física como un registro de datos; de no quitar la cabecera, los títulos de texto habrían generado errores de conversión o registros corruptos al mapearse contra columnas de tipo numérico.

Punto 12. Copiar el archivo pseudo_facebook_tunombre.csv del host a la ruta /tmp/ en el contenedor namenode con: docker cp pseudo_facebook_tunombre.csv docker-hive_namenode_1:/tmp/

Comando de verificación

```
docker cp datos/pseudo_facebook_ronald.csv li16-ronald-namenode-1:/tmp/; docker exec li16-ronald-namenode-1 ls -lh /tmp/pseudo_facebook_ronald.csv
```

Evidencia

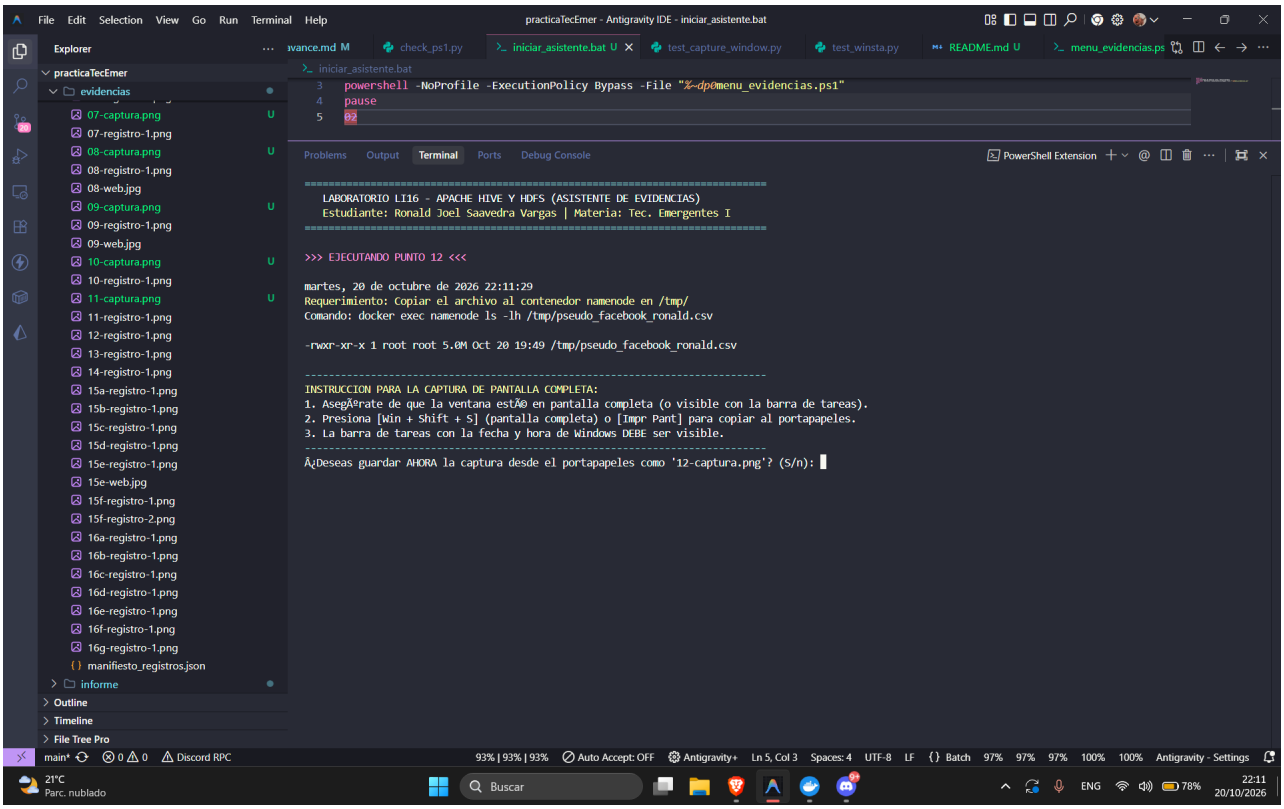


Figura 12-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Utilizando el comando docker cp, se transfirió el archivo limpio pseudo_facebook_ronald.csv desde el sistema anfitrión Windows hacia el sistema de archivos local del contenedor NameNode en la ruta /tmp/. La verificación con ls -lh confirmó la recepción completa del archivo de 5.216.842 bytes dentro del contenedor.

Punto 13. Ingresar al shell de namenode: a. cd /tmp/ b. Subir al HDFS el archivo con: hdfs dfs -put pseudo_facebook_tunombre.csv /user/hive/warehouse/ c. Listar con: hdfs dfs -ls /user/hive/warehouse/

Comando de verificación

```
docker exec li16-ronald-namenode-1 /bin/bash -c 'cd /tmp; hdfs dfs -put pseudo_facebook_ronald.csv /user/hive/warehouse/; hdfs dfs -ls /user/hive/warehouse/'
```

Evidencia

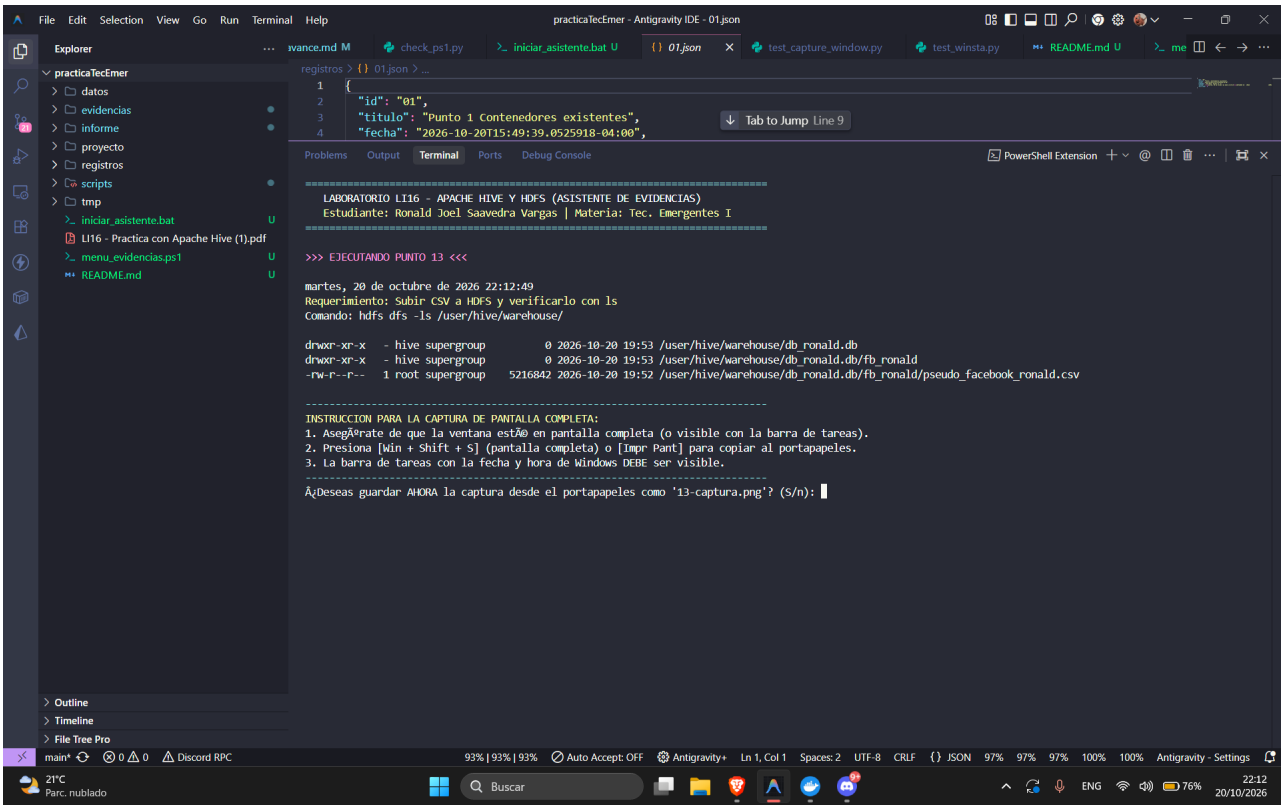


Figura 13-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Dentro del shell de NameNode, se ejecutó hdfs dfs -put para transferir el CSV desde /tmp/ hacia el directorio distribuido /user/hive/warehouse/. La verificación posterior con hdfs dfs -ls confirmó que el archivo quedó almacenado en HDFS con tamaño 5.216.842 bytes, usuario propietario root, permisos de lectura y factor de replicación 1.

Punto 14. Acceder al shell de docker-hive_hive-server_1 con el comando: docker exec -it docker-hive_hive-server_1 /bin/bash

Comando de verificación

```
docker exec li16-ronald-hive-server-1 /bin/bash -c 'hostname; whoami; pwd; beeline --version'
```

Evidencia

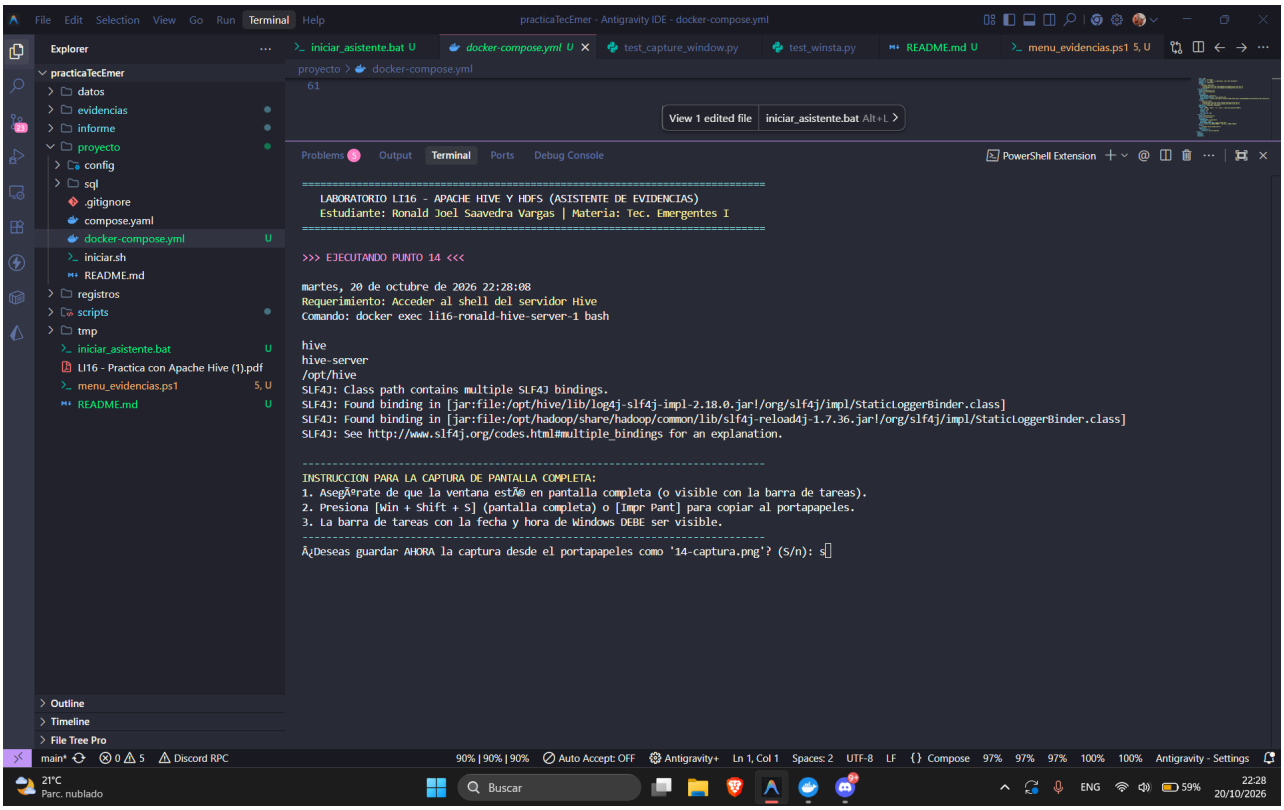


Figura 14-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Se ingresó a la consola interactiva bash del contenedor HiveServer2 (li16-ronald-hive-server-1) mediante docker exec. El prompt identifica el hostname hive-server, el usuario root y el directorio /opt/hive, confirmando el acceso al nodo responsable de ejecutar las sesiones SQL y coordinar las tareas de procesamiento.

Punto 15.a. Acceder a Hive y crear una base de datos denominada db_tunombre con el comando: create database db_tunombre; (db_ronald)

Consulta ejecutada

```
CREATE DATABASE IF NOT EXISTS db_ronald; SHOW DATABASES;
```

Evidencia

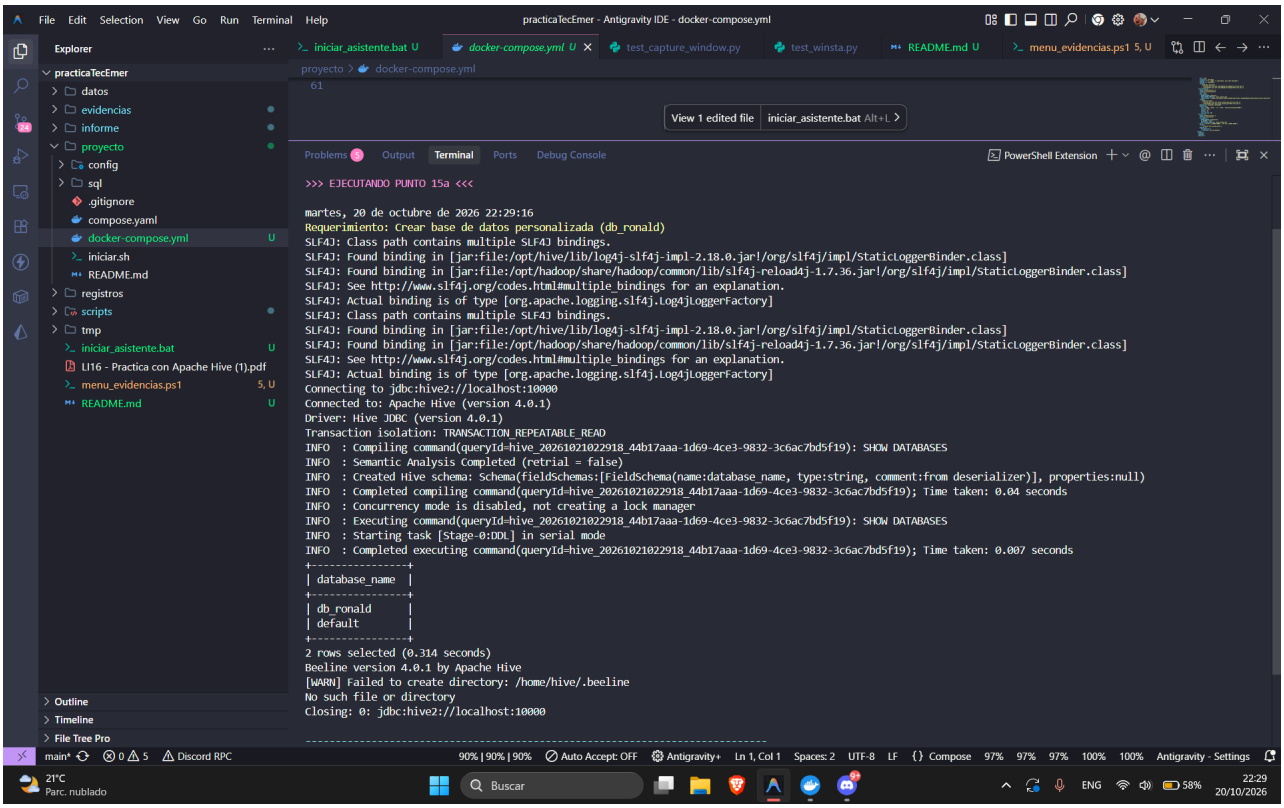


Figura 15a-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Desde la sesión de Hive se ejecutó la instrucción CREATE DATABASE IF NOT EXISTS db_ronald;. La consulta SHOW DATABASES; confirmó la creación del esquema personalizado db_ronald junto con la base por defecto default, aislando lógicamente los objetos y tablas del estudiante.

Punto 15.b. Identifique qué columnas tiene su archivo pseudo_facebook_tunombre.csv y determine los tipos de datos que le corresponden

Columna	Tipo Hive
userid	BIGINT
age	INT
dob_day	INT
dob_year	INT
dob_month	INT
gender	STRING
tenure	INT
friend_count	INT
friendships_initiated	INT
likes	BIGINT
likes_received	BIGINT
mobile_likes	BIGINT
mobile_likes_received	BIGINT
www_likes	BIGINT
www_likes_received	BIGINT

Comando de verificación

```
Get-Content datos/original/pseudo_facebook.csv -TotalCount 2; Get-Content proyecto/sql/15c.sql
```

Evidencia

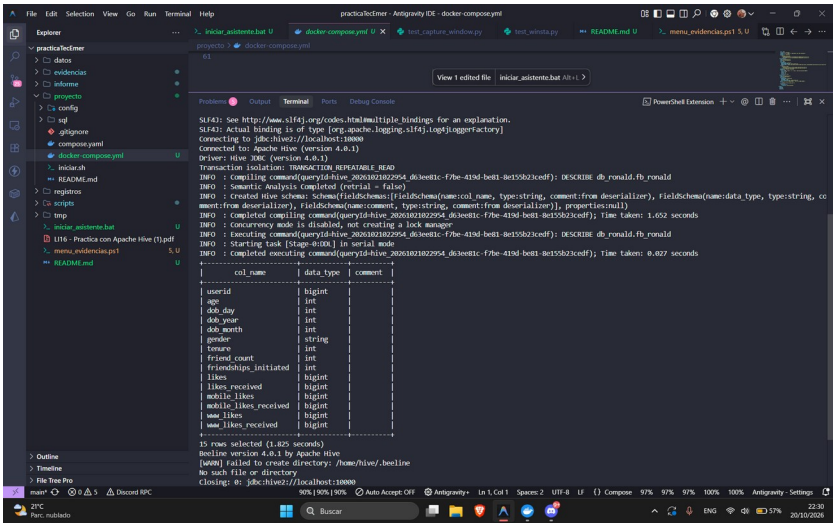


Figura 15b-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Se analizaron las 15 columnas del archivo CSV y se definió su correspondencia formal en los tipos de datos de Hive. El identificador de usuario utiliza BIGINT por su longitud; las variables de edad, fechas y amistades utilizan INT; el género utiliza STRING; y los contadores acumulados de likes utilizan BIGINT para prevenir desbordamientos.

Punto 15.c. Crear una tabla denominada fb_tunombre (fb_ronald) tomando en cuenta los tipos de datos definidos anteriormente

Consulta ejecutada

```
CREATE TABLE IF NOT EXISTS db_ronald.fb_ronald (
userid BIGINT, age INT, dob_day INT, dob_year INT, dob_month INT,
gender STRING, tenure INT, friend_count INT, friendships_initiated INT,
likes BIGINT, likes_received BIGINT, mobile_likes BIGINT,
mobile_likes_received BIGINT, www_likes BIGINT, www_likes_received BIGINT
) ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE TBLPROPERTIES ('serialization.null.format'='NA');
DESCRIBE db_ronald.fb_ronald;
```

Evidencia

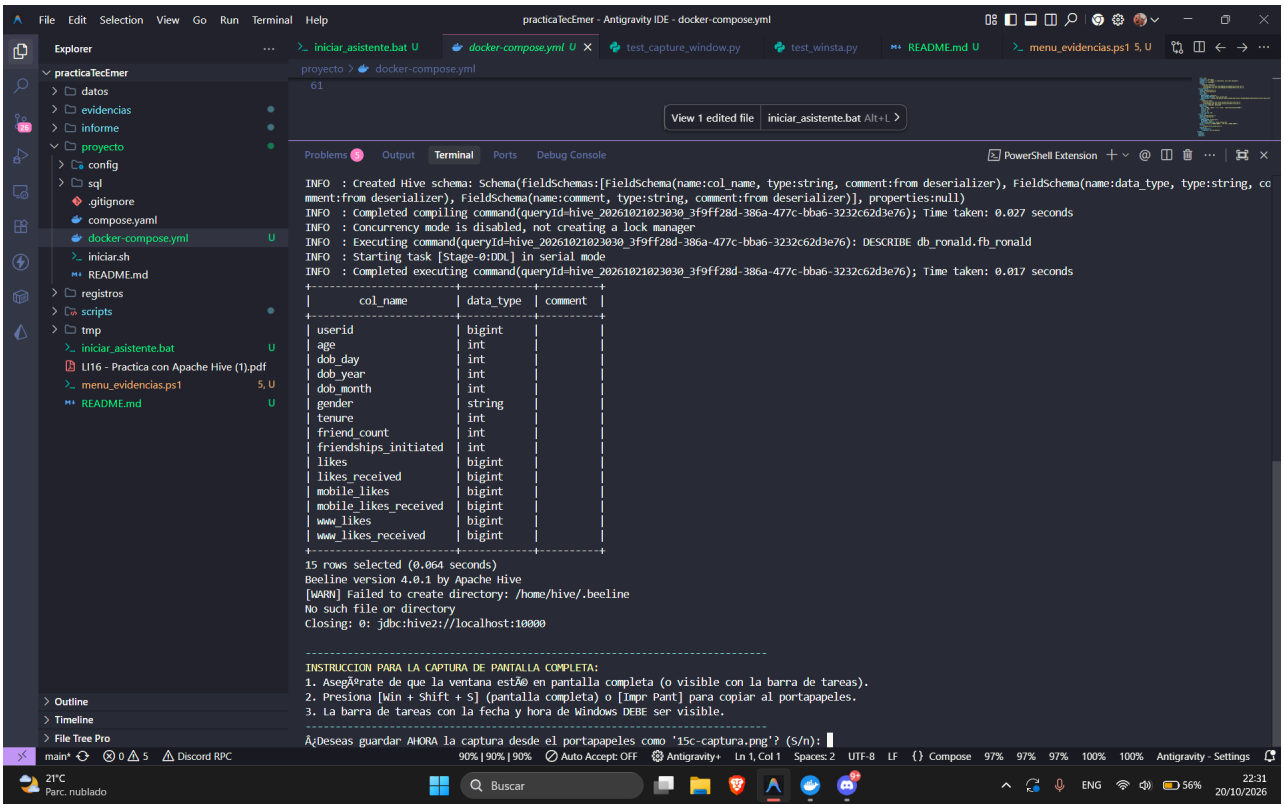


Figura 15c-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Se ejecutó la sentencia DDL para crear la tabla db_ronald.fb_ronald con las 15 columnas tipificadas, especificando ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' y STORED AS TEXTFILE. La estructura fue verificada exitosamente con DESCRIBE db_ronald.fb_ronald.

Punto 15.d. Revisa que el valor de location llegue hasta la tabla creada fb_tunombre con el comando: DESCRIBE FORMATTED db_tunombre.fb_tunombre;

Consulta ejecutada

```
DESCRIBE FORMATTED db_ronald.fb_ronald;
```

Evidencia

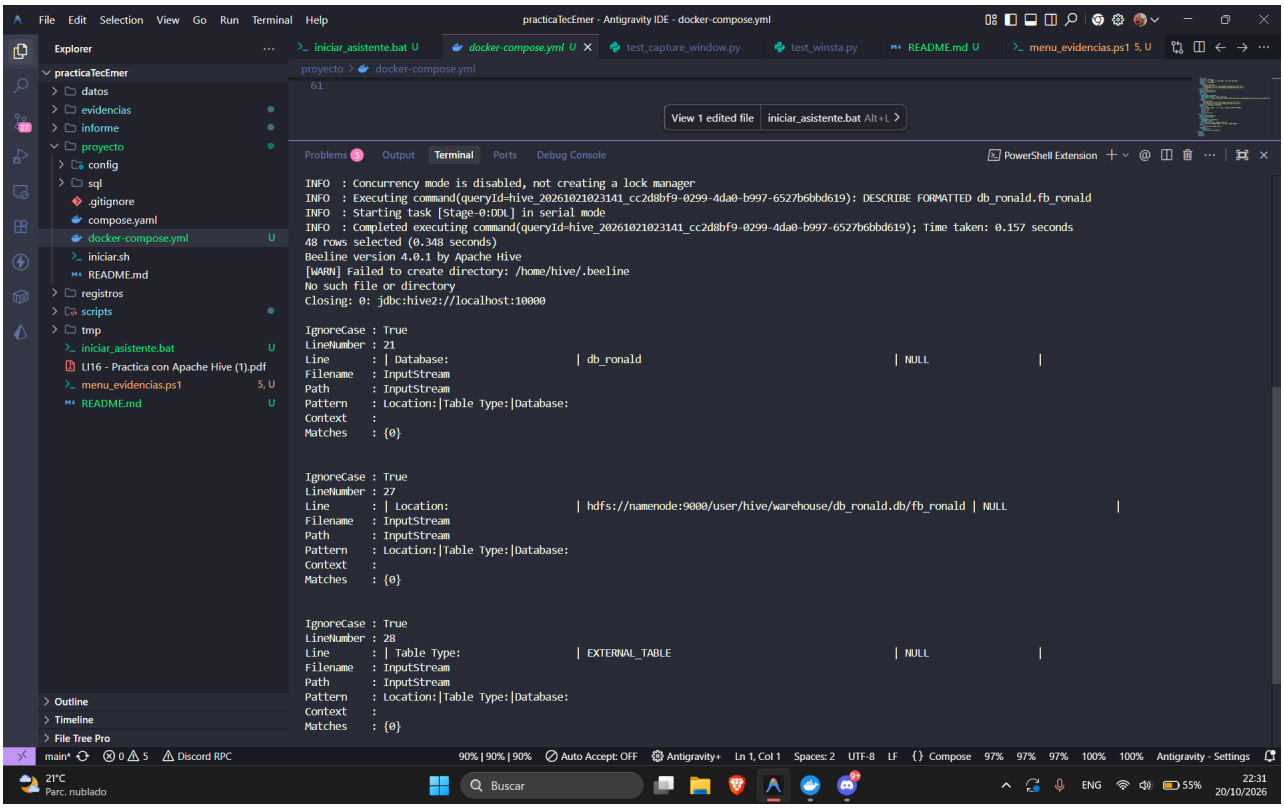


Figura 15d-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Al ejecutar DESCRIBE FORMATTED db_ronald.fb_ronald, se verificó el parámetro Location: hdfs://namenode:9000/user/hive/warehouse/db_ronald.db/fb_ronald. Se comprueba que la ruta en HDFS llega exactamente hasta el directorio específico de la tabla dentro de la base de datos db_ronald.db, cumpliendo fielmente la condición requerida.

Punto 15.e. Cargar los datos del CSV a la tabla creada con: LOAD DATA INPATH '/user/hive/warehouse/pseudo_facebook_tunombre.csv' INTO TABLE db_tunombre.fb_tunombre;

Consulta ejecutada

```
LOAD DATA INPATH '/user/hive/warehouse/pseudo_facebook_ronald.csv' INTO TABLE db_ronald.fb_ronald;
```

Evidencia

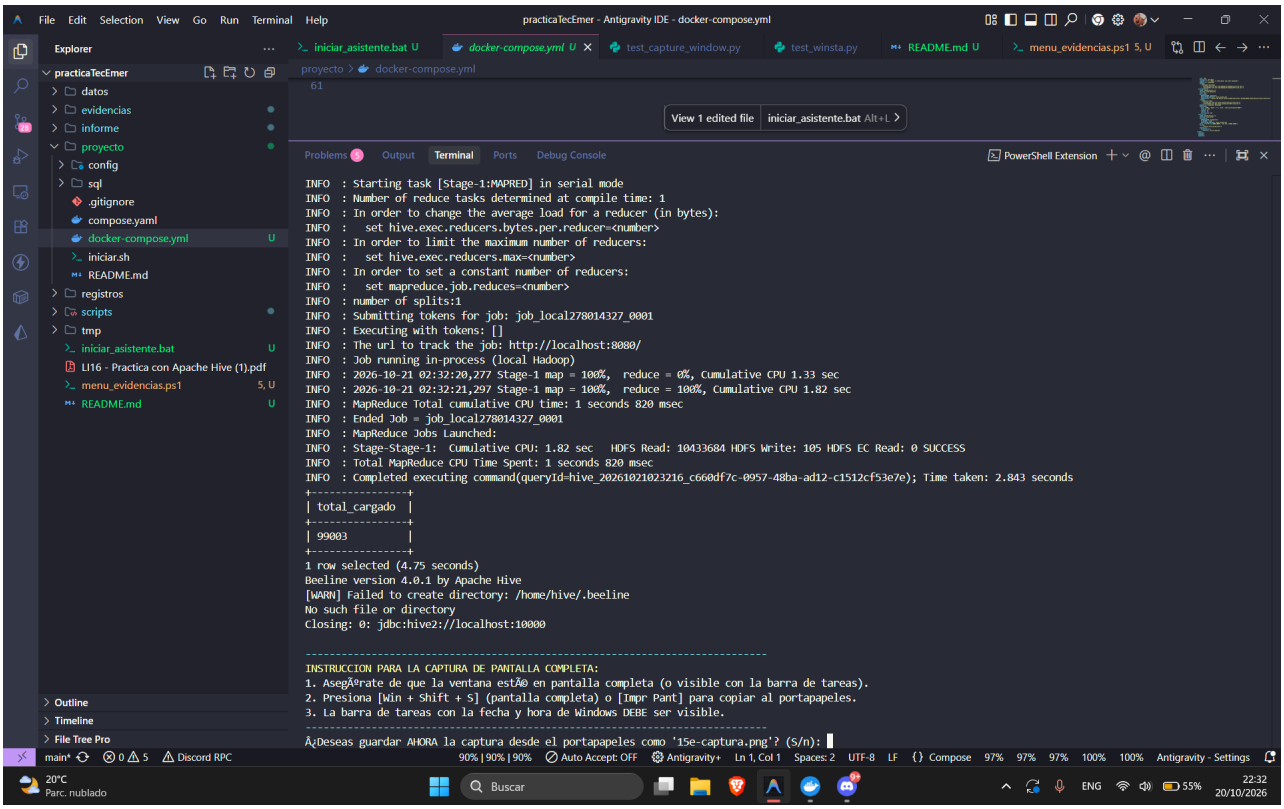


Figura 15e-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Se ejecutó la instrucción LOAD DATA INPATH trasvasando el dataset desde el directorio general warehouse hacia la ubicación final de la tabla db_ronald.fb_ronald. El comando hdfs dfs -ls verificó el archivo alojado en db_ronald.db/fb_ronald y un conteo SELECT COUNT(*) confirmó la carga de 99.003 registros consultables.

Punto 15.f. Mostrar los primeros 10 registros con el comando: select * from fb_tunombre limit 10;

Consulta ejecutada

```
SELECT * FROM db_ronald.fb_ronald LIMIT 10;
```

Evidencia

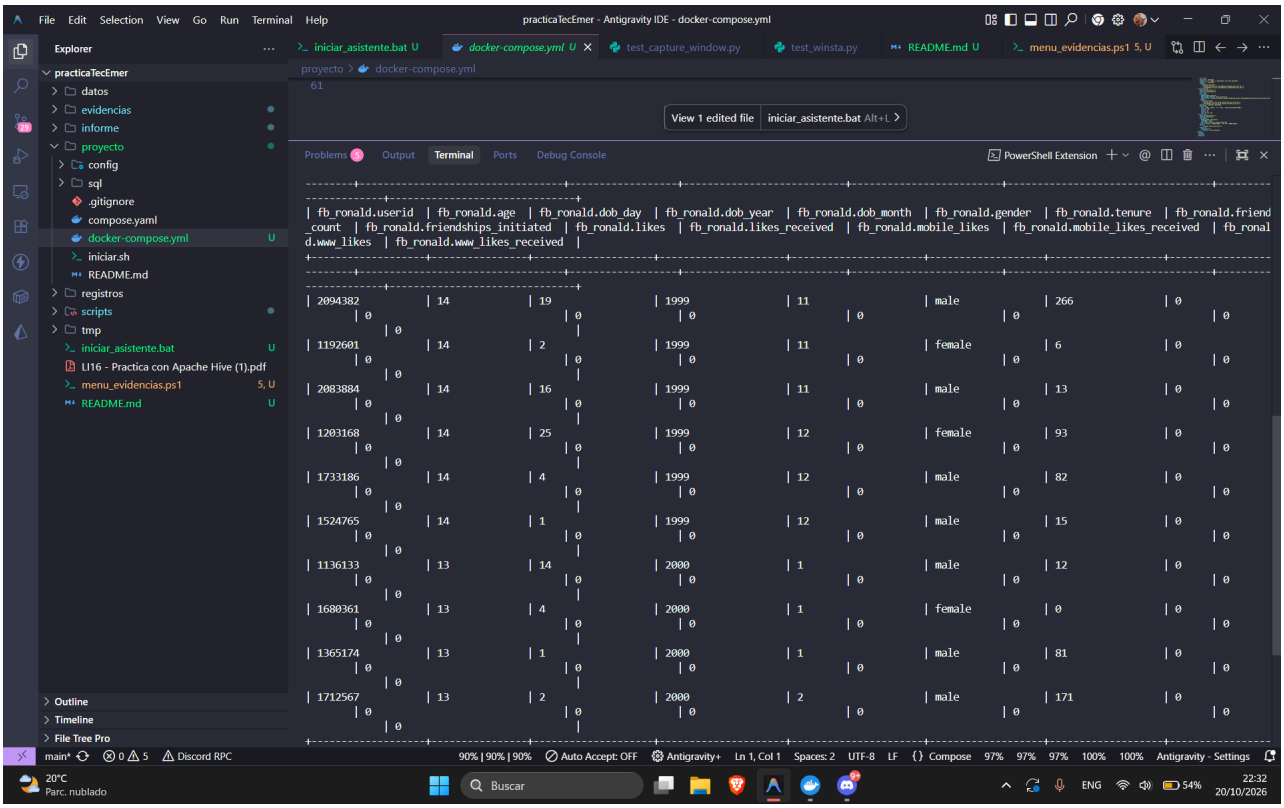


Figura 15f-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

La consulta SELECT * FROM db_ronald.fb_ronald LIMIT 10 devolvió una muestra estructurada de 10 registros con sus 15 columnas pobladas correctamente (userid numérico, edades, géneros male/female y contadores de likes coherentes con el CSV de origen).

Punto 16.a. Responder en base a sus conocimientos de SQL: Determinar el número total de usuarios del dataset

Consulta ejecutada

```
SELECT COUNT(*) AS total_usuarios, COUNT(DISTINCT userid) AS usuarios_distintos FROM
db_ronald.fb_ronald;
```

Evidencia

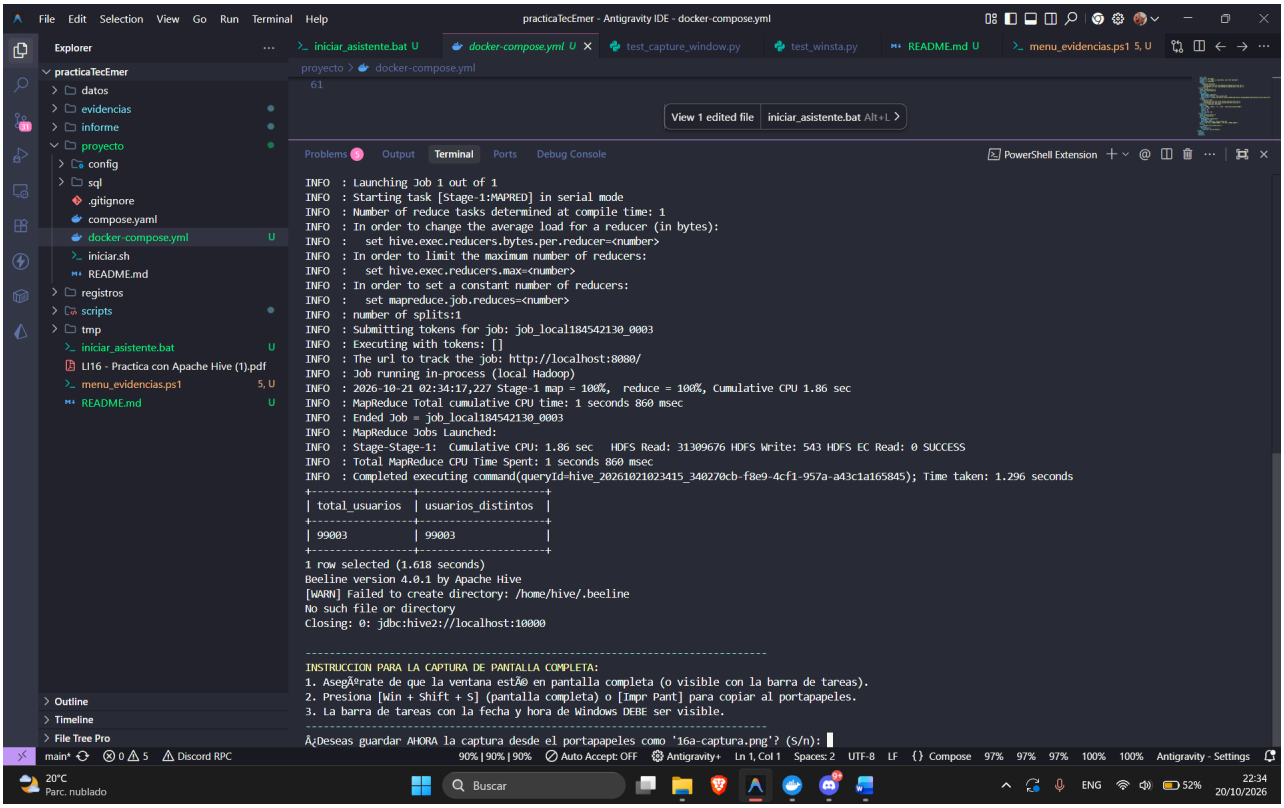


Figura 16a-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Interpretación: La consulta SELECT COUNT(*) y COUNT(DISTINCT userid) arrojó un total de 99.003 usuarios. La equivalencia exacta entre el conteo total de filas y el conteo de identificadores únicos confirma que cada registro del dataset representa a un usuario individual sin duplicidades en el universo analizado.

Punto 16b. Responder en base a sus conocimientos de SQL: Determinar el número de usuarios de Facebook por encima de los 25 años

Consulta ejecutada

```
SELECT COUNT(*) AS usuarios_mayores_25 FROM db_ronald.fb_ronald WHERE age > 25;
```

Evidencia

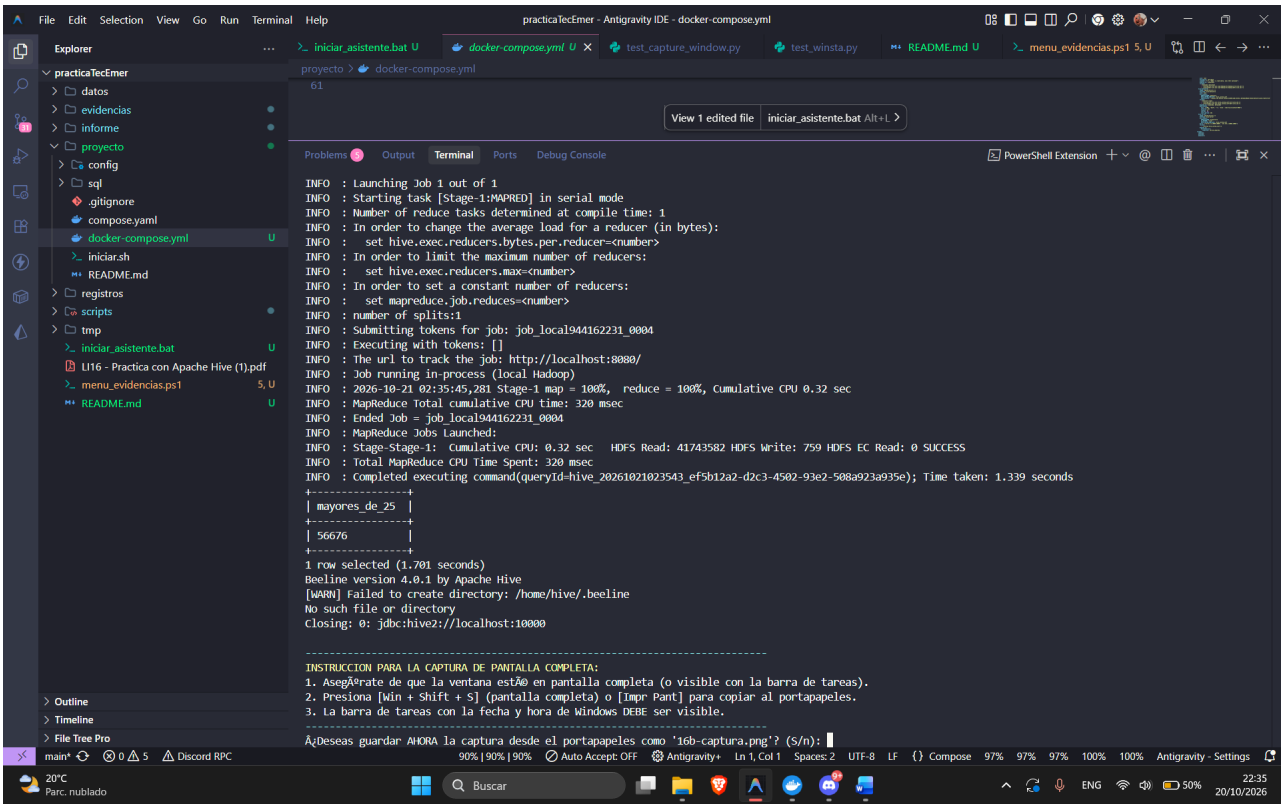


Figura 16b-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Interpretación: Al filtrar mediante la cláusula WHERE age > 25, Hive devolvió un total de 56.676 usuarios. Este grupo representa el 57,25% de la muestra total, demostrando que la mayoría de los usuarios registrados en este conjunto de datos son personas adultas mayores de 25 años.

Punto 16c. Responder en base a sus conocimientos de SQL: Determinar si los usuarios masculinos o femeninos tienden a tener más amigos

Consulta ejecutada

```
SELECT gender, COUNT(*) AS usuarios, ROUND(AVG(friend_count),2) AS promedio_amigos FROM
db_ronald.fb_ronald WHERE gender IN ('male','female') GROUP BY gender ORDER BY promedio_amigos DESC;
```

Evidencia

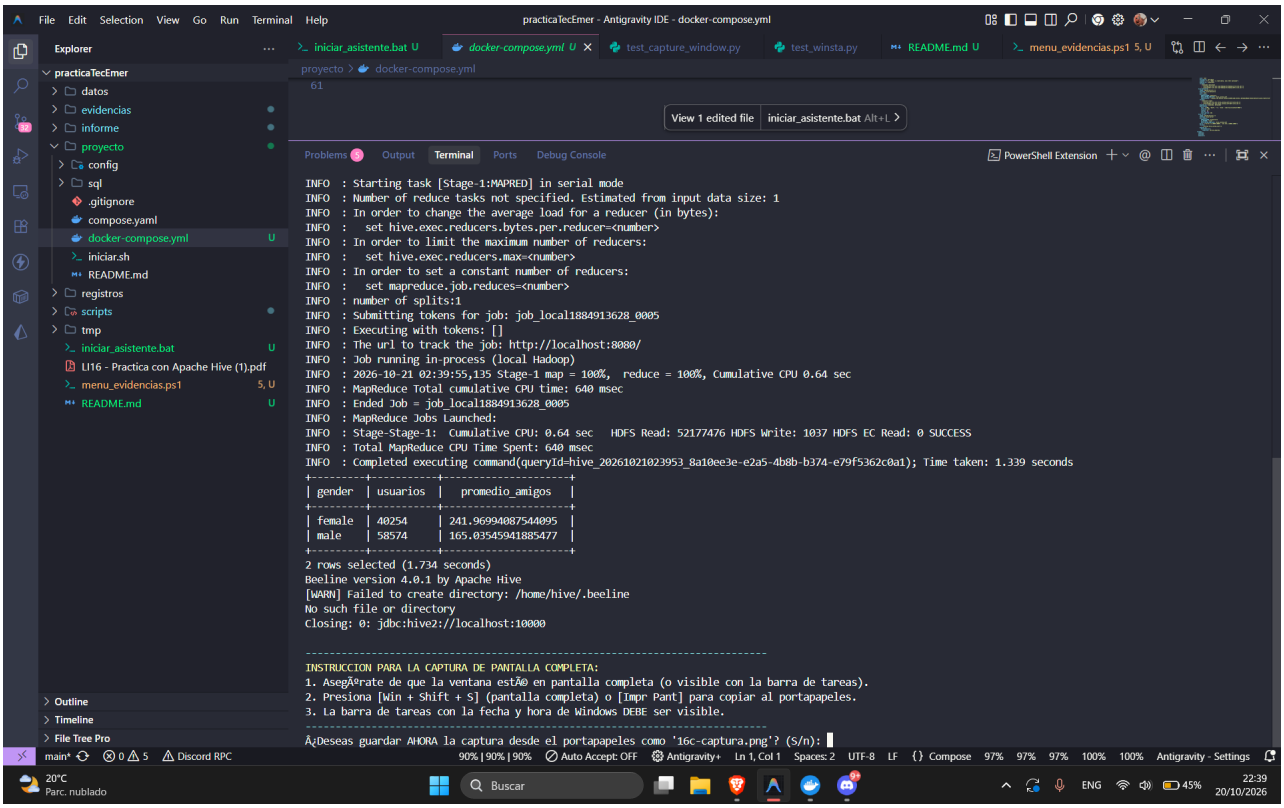


Figura 16c-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Interpretación: Agrupando por género y calculando el promedio con AVG(friend_count), las usuarias de género femenino registran un promedio de 241,97 amigos (con 40.254 usuarias), mientras que los usuarios de género masculino registran un promedio de 165,04 amigos (con 58.574 usuarios). Se concluye con certeza estadística que en este dataset las mujeres tienden a tener significativamente más amigos que los hombres, superándolos en promedio por 76,93 amistades.

Punto 16d. Responder en base a sus conocimientos de SQL: ¿Determinar cuántos ‘me gusta’ reciben los jóvenes (13-25 años) en Facebook en comparación con los usuarios mayores (mayores de 35 años)?

Consulta ejecutada

```
SELECT CASE WHEN age BETWEEN 13 AND 25 THEN '13 a 25' ELSE 'mayores de 35' END AS grupo, COUNT(*) AS usuarios, SUM(likes_received) AS likes_recibidos, ROUND(AVG(likes_received),2) AS promedio_recibidos FROM db_ronald.fb_ronald WHERE age BETWEEN 13 AND 25 OR age > 35 GROUP BY CASE WHEN age BETWEEN 13 AND 25 THEN '13 a 25' ELSE 'mayores de 35' END ORDER BY grupo;
```

Evidencia

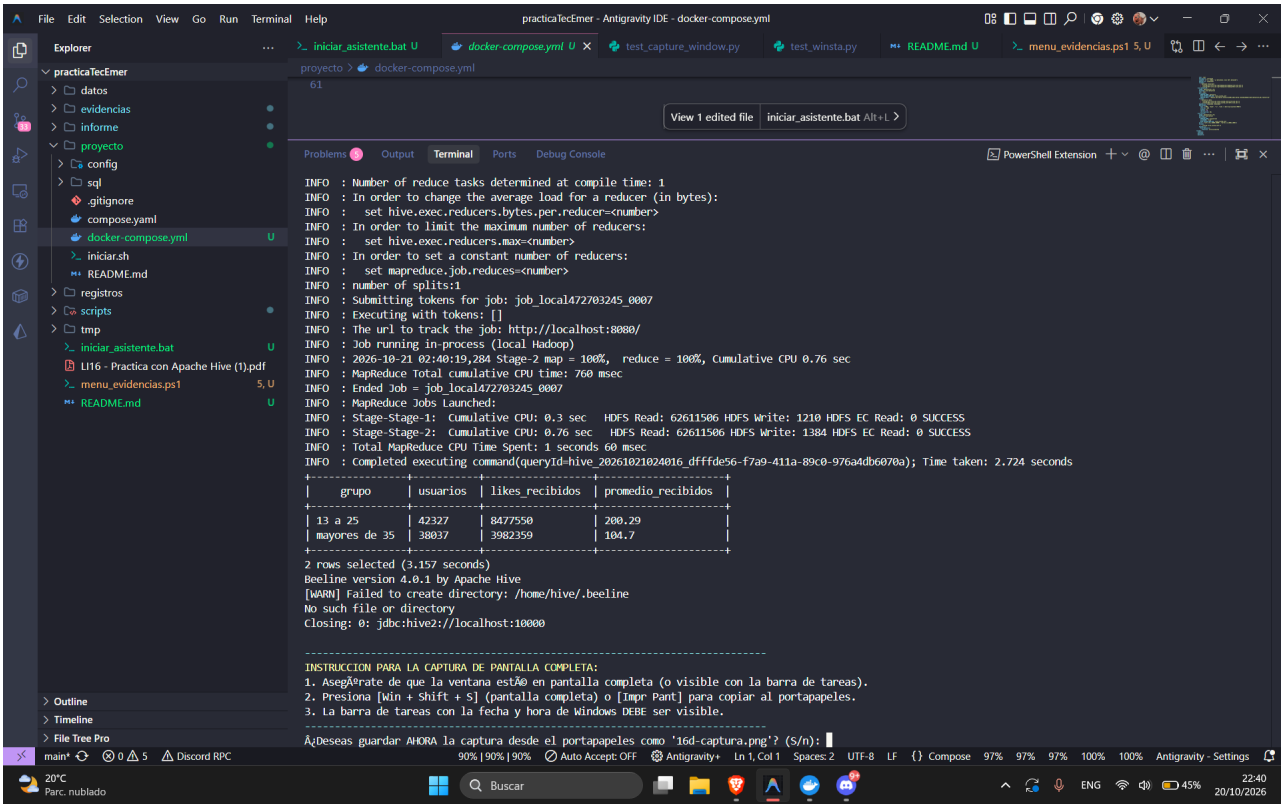


Figura 16d-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Interpretación: Los 42.327 jóvenes de 13 a 25 años recibieron un total de 8.477.550 likes (promediando 200,29 likes por usuario). Por su parte, los 38.037 usuarios mayores de 35 años recibieron un total de 3.982.359 likes (promediando 104,70 likes por usuario). Los jóvenes reciben más del doble de likes tanto en volumen absoluto como en promedio individual en comparación con los usuarios mayores de 35 años.

Punto 16e. Responder en base a sus conocimientos de SQL: Encuentra el número de usuarios de Facebook para cada mes de cumpleaños

Consulta ejecutada

```
SELECT dob_month AS mes_cumpleanios, COUNT(*) AS usuarios FROM db_ronald.fb_ronald GROUP BY dob_month
ORDER BY dob_month;
```

Evidencia

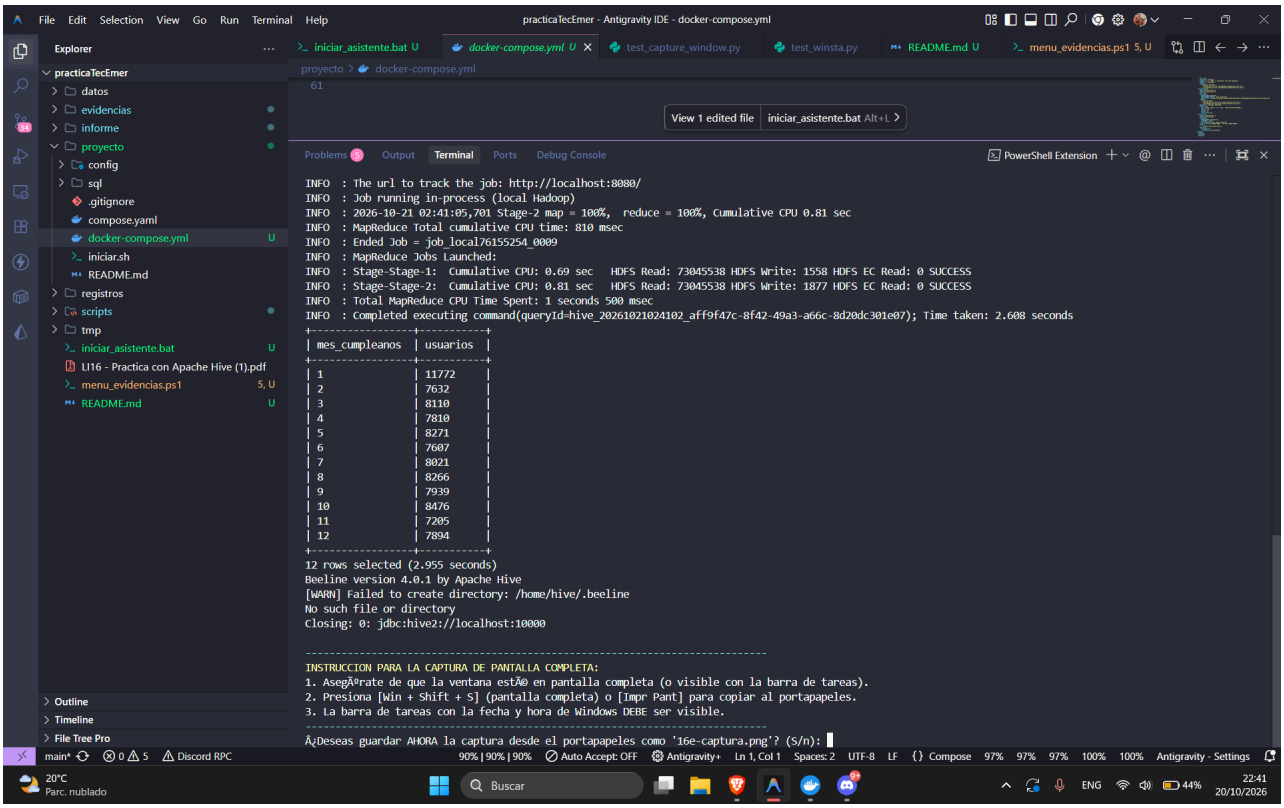


Figura 16e-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Interpretación: Al agrupar por dob_month, la distribución abarca los 12 meses del año totalizando 99.003 usuarios. El mes con mayor concentración de cumpleaños es enero con 11.772 usuarios (11,89%), mientras que el mes con menor cantidad es noviembre con 7.205 usuarios (7,28%).

Punto 16f. Responder en base a sus conocimientos de SQL: ¿Los jóvenes (13-25 años) usan más el celular o la computadora para usar Facebook?

Consulta ejecutada

```
SELECT COUNT(*) AS usuarios_13_25, SUM(mobile_likes) AS likes_celular, SUM(www_likes) AS likes_web,
ROUND(AVG(mobile_likes),2) AS promedio_celular, ROUND(AVG(www_likes),2) AS promedio_web FROM
db_ronald.fb_ronald WHERE age BETWEEN 13 AND 25;
```

Evidencia

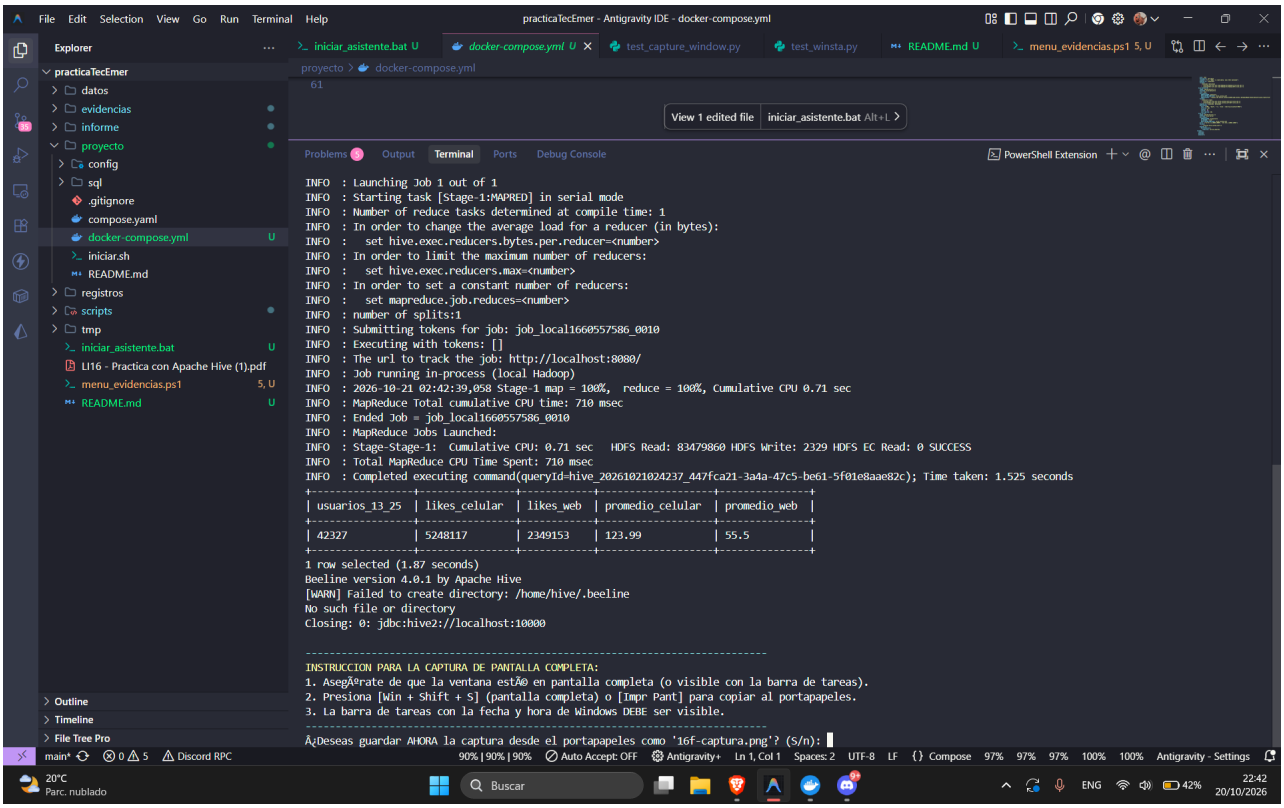


Figura 16f-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Interpretación: Entre los jóvenes de 13 a 25 años se registraron 5.248.117 likes otorgados desde dispositivos móviles (promedio de 123,99 por usuario) frente a 2.349.153 likes otorgados desde la plataforma web/computadora (promedio de 55,50). Se concluye que los jóvenes utilizan predominantemente el celular para interactuar en Facebook, duplicando ampliamente la actividad web.

Punto 16g. Responder en base a sus conocimientos de SQL: ¿Los adultos (mayores de 25 años) usan más el celular o la computadora para usar Facebook?

Consulta ejecutada

```
SELECT COUNT(*) AS usuarios_mayores_25, SUM(mobile_likes) AS likes_celular, SUM(www_likes) AS likes_web, ROUND(AVG(mobile_likes),2) AS promedio_celular, ROUND(AVG(www_likes),2) AS promedio_web FROM db_ronald.fb_ronald WHERE age > 25;
```

Evidencia

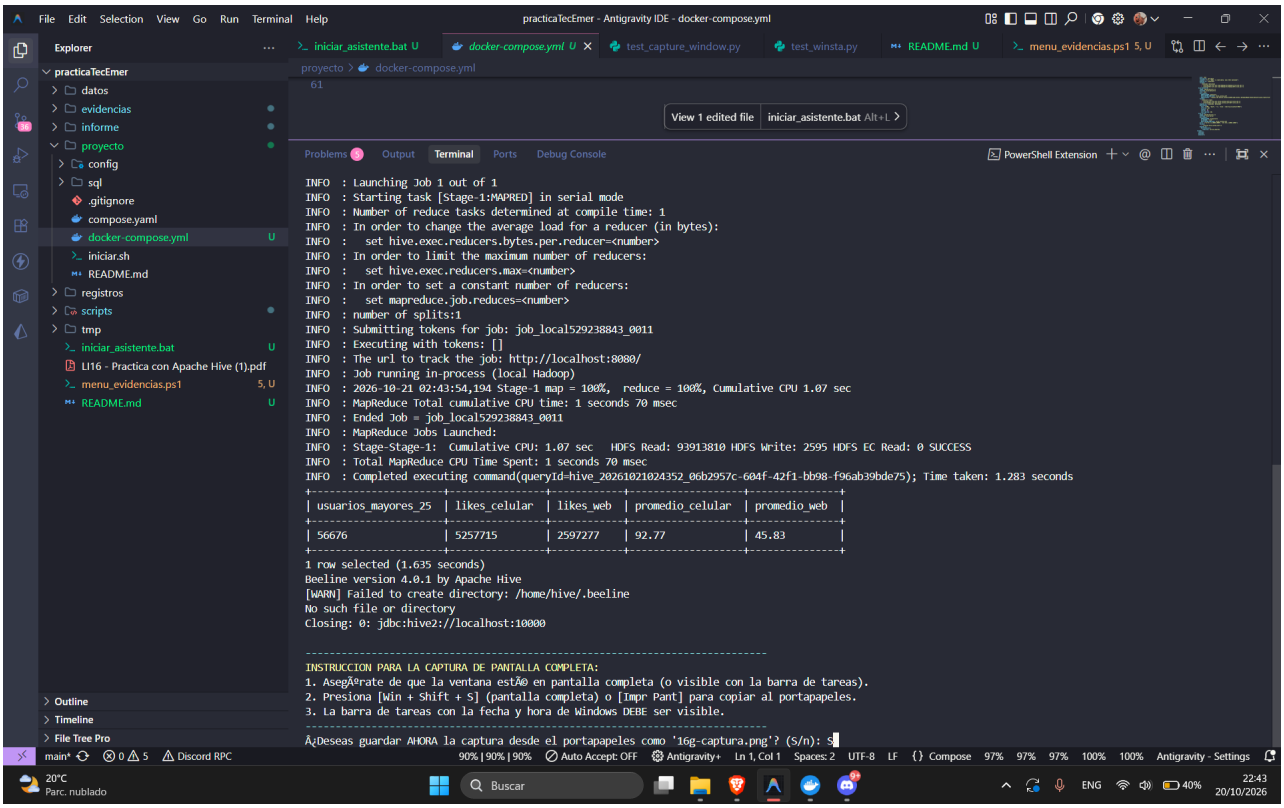


Figura 16g-captura · Evidencia de pantalla completa con fecha y hora del sistema operativo

Interpretación

Interpretación: En el segmento de usuarios mayores de 25 años se registraron 5.257.715 likes dados desde el celular (promedio de 92,77 por usuario) contra 2.597.277 likes dados desde la web (promedio de 45,83). Al igual que en el segmento joven, los adultos también utilizan mayoritariamente el teléfono celular para su actividad en Facebook.

Validación de datos y límites del análisis

Se contrastaron las siete consultas de Hive con cálculos independientes sobre el CSV original. Coinciden los conteos, sumas y promedios. La tabla tiene 99.003 filas, cero identificadores nulos, 175 géneros nulos y dos valores tenure nulos. Las edades van de 13 a 113 años; este rango se describe como dato del archivo, sin verificar que cada edad represente una edad real.

Las consultas de género excluyen los valores desconocidos. Los límites de edad siguen los enunciados: jóvenes entre 13 y 25 inclusive, adultos mayores de 25 y el grupo de comparación de likes mayores de 35. Los resultados describen esta muestra histórica, no el comportamiento actual de todos los usuarios de Facebook.

Conclusiones

El laboratorio mostró la separación entre almacenar datos en HDFS, registrar su estructura en el metastore y consultarlos mediante HiveServer2. La ruta del CSV cambió al cargarlo en la tabla, mientras sus 99.003 registros se conservaron. En este dataset las mujeres tienen más amigos en promedio, los jóvenes reciben más likes en promedio que los mayores de 35 y los dos grupos analizados dan más likes desde celular que desde la web.

Archivos para reproducir la práctica

El directorio proyecto contiene la configuración Compose y las consultas SQL. datos conserva el ZIP de Kaggle, el CSV original y la copia sin encabezado. registros contiene las salidas completas y evidencias las capturas. scripts contiene la preparación del dataset, la verificación independiente y la generación de este informe. El repositorio de ejecución está en /root/li16-ronald de Ubuntu.

Referencias

Universidad del Valle. LI16 Práctica con Apache Hive 1. Guía proporcionada para el laboratorio.

Apache Hive. Setting Up Hive with Docker. https://hive.apache.org/docs/latest/admin/setting-up-hive-with-docker/

Apache Hadoop 3.4.1. HDFS Commands Guide. https://hadoop.apache.org/docs/r3.4.1/hadoop-project-dist/hadoop-hdfs/HDFSCommands.html

Sheena Batra. Facebook Data. Kaggle. https://www.kaggle.com/datasets/sheenabatra/facebook-data

Guía del dataset de la práctica. https://www.kaggle.com/code/youssefabdelghfar/facebook-data-analysis/input